

Durability: the journal, the snapshot, and what "no loss" means

The win: you can explain, to an interviewer or to yourself, why an acknowledgement may only be given after a durable append and what goes wrong one line earlier; how at-least-once delivery plus a unique key keyed by sender becomes exactly-once across a restart; why a room that cannot write says so to everyone, acknowledges nothing, and reads the log before it writes again; why a snapshot is written once with a conditional put and named in Postgres only after it exists; and what a chaos run has to record for "no acknowledged op is lost" to be a checked fact rather than a hope; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests (the room's journal, read-only and recovery rules; peer-client's hold; the snapshotter; the chaos checker on synthetic ledgers). The integration suites (`packages/db_journal.int`, `sync_journal.int`, `read-only.int`, `snapshot.int` including the 10,000-row open under 2 s, and the E6.1b additions to the worker's `push.test` and `git.int`) and both chaos checks (`postgres-down-read-only`, `kill-sync-no-loss`) are written and committed and *have not been run*: the host was too loaded to run Docker suites beside other work. They run on `build/epics` before anything merges to main. Where this page says "measured", it was measured in a learning test or an earlier epic; where it says "the test asserts", the test exists and is unrun. Section 7 is the by-hand demo to run first.

How to use this page. Every code block is cut from the repo at commit `f21d6d2`, so what you read is what is committed. Sections 1, 3 and 4 are the ones a distributed-systems interview will dig into; section 5 is the one that turns the promise into a checked fact.

```
./init.sh # now also MinIO on 127.0.0.1:9005; sync makes its bucket
at start
pnpm exec vitest run apps/sync/src/room.test.ts apps/sync/src/snapshots.test.ts packages/peer-client/src/peer.test.ts
scripts/chaos/no-loss.test.ts # the pure halves
docker compose kill --signal SIGKILL sync && docker compose start sync # the F18 fault, by hand; the
canvas reconnects unaided
```

Try it, once the stack is up: section 7 walks the two faults by hand, then the suites and the chaos runs, with the exact commands and what each should show.

The map: what durability means here

- 1 · The journal: durable before anyone hears of it
- 2 · Exactly once, again: the resend, the forgotten op, and the id that never returns
- 3 · The database is gone: read-only, and the recovery that reads before it writes
- 4 · Snapshots: a cache of the log, written once, named after it exists
- 5 · Proving the promise: a ledger, the journal, and two faults on the real stack
- 6 · Testing durability without a database, and then with one
- 7 · Durability by hand: what to run once the stack is up
- 8 · What the type system could not see, epic 6 edition

Your turn: three drills

Recall, then test yourself

The map: what durability means here

Epics 2 to 5 built a room that orders every edit and three kinds of peer that write through it. None of it survived a crash: F8 saved the document when a room went idle, and a sync process killed mid-edit lost everything since. Epic 6 makes one promise and keeps it under two faults. **The promise (F18): no acknowledged edit is ever lost, and no edit is ever applied twice.** The faults: the sync process dies (`kill -9` , no goodbye), and the database goes away (stopped, or paused so that every query hangs). A third feature (F19) makes the promise affordable: a document with ten thousand journaled edits still opens in under two seconds.

STEP	WHERE	AND	WHAT HAPPENS
1	packages/db/migrations/0012_op_journal.sql	packages/db/src/index.ts	the op journal: an append only table with two unique keys told apart by name; <code>append</code> , <code>find</code> , <code>everAdded</code> , <code>since</code>
2	apps/sync/src/room.ts		the room's step 3 becomes <i>durable, then applied, then announced</i> ; a forgotten resend is asked of the journal; keystone 4 is asked of the journal
3	apps/sync/src/room.ts	packages/contracts/src/index.ts	a failed append turns the room read-only: one <code>status</code> to everyone, no acknowledgement to anyone, and <code>recover()</code> replays the journal before saying writable
4	apps/sync/src/server.ts	packages/peer-client/src/peer.ts	a journal call bounded by timeout; a retry loop that calls <code>recover()</code> ; a client that holds its edits, refuses new ones, and sends the held ones on "writable"
5	apps/web/src/Canvas.tsx	apps/worker/src/tools.ts	apps/worker/src/push.ts every kind of peer hears read-only through one contract: the canvas shows an alert, the AI's tool call is refused, the git peer's ever waits
6	apps/sync/src/snapshots.ts	packages/db/migrations/0013_snapshot_seq.sql	snapshots to MinIO on a cadence and on last leave, keyed so that string order is numeric order, written once (<code>IfExists</code>), named in Postgres only after they exist
7	apps/sync/src/server.ts		opening a document: the newest snapshot, then on the journal rows after it; F8's idle save is deleted
8	scripts/chaos/no-loss.ts	scripts/chaos/kill-sync-no-loss.ts	scripts/chaos/postgres-down-read-only.ts: the promise as a pure check over a ledger and the journal, and two runs that break the real stack

Four ideas carry the epic. **A write-ahead log** (section 1): the op is written to a place that survives the process *before* the process tells anyone, so the acknowledgement a peer sees is a fact about the database, not about memory. **At-least-once plus an idempotent resend is exactly-once** (section 2): a client that never heard its answer sends the same op again with the same id, and the journal's unique key turns the second sending into the first answer. **Fail visibly, recover from the log** (section 3): when the log cannot be written the room stops acknowledging and says so; when it can, the room reads what landed without it before it writes again. **A snapshot is a cache of the log** (section 4): losing one loses nothing, so it is written best-effort and never retried, and the pointer to it is written only after the object exists.

THIS REPO	JAVA WORLD	PYTHON WORLD
an append-only Postgres table as the write-ahead log, the room as the single writer	Kafka as the log with one producer per partition; or an event store (Axon, EventStoreDB)	the same table with psycopg; or an event store
dedupe by <code>(document, sender, opId)</code> unique key, answered with the original row	an idempotency key table in the same transaction as the write	the same
read-only as a broadcast state with a replay-before-writable recovery	a circuit breaker that half-opens, plus a catch-up read	the same idea; <code>pybreaker</code> for the breaker
snapshots as gzipped objects with a conditional put and a pointer row	an S3 checkpoint with <code>ifNoneMatch("*")</code> in SDK v2; Kafka Streams' state store checkpoint	boto3 <code>put_object(IfNoneMatch="*")</code>

1 · The journal: durable before anyone hears of it

The room already had a persist step between validating an op and applying it (lesson 2 awaited it so that ordering under a slow database could be tested). Epic 6 gives that step a table and a contract, and the room's whole promise follows from *where* the step sits: an op is written to `op_journal`, and only if the write succeeds is it applied to the in-memory document, remembered, and broadcast. The sender's own copy of the broadcast is its acknowledgement. So an acknowledged op is, by construction, a row that survives the process.

FROM `packages/db/migrations/0012_op_journal.sql`

```
-- E6.1a (F18): every op the room accepted, written BEFORE anyone hears of it. Append-only: rows are never
-- updated. Two unique keys, told apart by NAME when an insert breaks one (learning-tests/postgres):
-- op_journal_seq: one op per number. Broken = a second writer is numbering this document: refuse.
-- op_journal_op: one row per sender's opId. Broken = a resend the room had forgotten: answer with the
--               original row. Keyed by the SENDER, as the room's own memory is: every broadcast shows
--               every opId to every peer, so another peer's opId is not a resend.
create table op_journal (
  document_id uuid not null,
  org_id      uuid not null,
  seq        bigint not null check (seq >= 1),
  op_id      uuid not null,
  actor_kind text not null check (actor_kind in ('user', 'agent', 'git')),
  actor_id   text not null check (length(actor_id) between 1 and 200),
  run_id     text check (length(run_id) between 1 and 200),
  op         jsonb not null,
  created_at timestamptz not null default now(),
  constraint op_journal_seq primary key (document_id, seq),
  constraint op_journal_op unique (document_id, actor_id, op_id),
  foreign key (document_id, org_id) references documents (id, org_id) on delete cascade
);
-- Keystone 4: a node id that was ever added is never added again. The room asks this before each add.
create index op_journal_added on op_journal (document_id, (op ->> 'nodeId')) where op ->> 'type' = 'add_node';
```

Read the two constraints as two different *facts*. `op_journal_seq` on `(document_id, seq)` says one op per number: if an insert breaks it, someone else is numbering this document, which must never happen with one room per document, and the right answer is to refuse and stop writing. `op_journal_op` on `(document_id, actor_id, op_id)` says one row per sender's opId: if an insert breaks it, the room forgot a resend, and the right answer is the original row. Same SQLSTATE (23505, unique violation) for both, so the code cannot tell them apart by *code*: it reads the *constraint name* the driver exposes (`learning-tests/postgres` , finding 1).

FROM `learning-tests/postgres/test.ts`

```
// 1. Duplicate insert violating a unique constraint fails with code === '23505' and the
// error exposes which constraint via `.constraint` (and `.table`, `.detail`, `.schema`).
// CONFIRMED. Both the (document_id, seq) and (document_id, op_id) unique constraints
// produced distinct `.constraint` names, so the app can tell a duplicate seq from a
// duplicate op_id by string-comparing `err.constraint`.
```

FROM `packages/db/src/index.ts`

```
/**
 * E6.1a: journals an accepted op, BEFORE anyone hears of it. Undefined: written. An op: this sender's
 * opId was journaled already, and that is what it became (a resend the room had forgotten). Rejects
 * when the op is not durable: the database is away, the seq is taken (a second writer), the document is gone,
 * or (E7.3) `claim` is no longer the document's: `Fenced`. No claim: only a document never claimed takes it.
 */
append(orgId: string, documentId: string, op: SequencedOp, claim?: string): Promise<SequencedOp | undefined>;
```

FROM packages/db/src/index.ts

```
async append(orgId, documentId, { seq, opId, actor, op }, claim) {
  if (!isId(orgId) || !isId(documentId)) throw new Error("journal: not a document id");
  try {
    // From the document's row, so that a document of another org (or one deleted meanwhile) takes no op.
    // The fence is in the same statement (F22): the row must still name this room's claim, and FOR UPDATE
    // locks it, so a claim either waits for this insert (and its reader sees the row) or comes first (and
    // Postgres re-checks the WHERE against the claimed row: nothing is inserted). Never read-then-write.
    const written = await pool.query(
      `insert into op_journal (document_id, org_id, ${JOURNAL_COLUMNS}) select id, org_id, $3, $4, $5, $6, $7, $8 from documents
where org_id = $1 and id = $2 and fence_claim is not distinct from $9 for update`,
      [orgId, documentId, seq, opId, actor.kind, actor.id, actor.runId ?? null, JSON.stringify(op), claim ?? null],
    );
    if (written.rowCount !== 1) {
      // Only telling the two refusals apart, after the fact: nothing was written either way.
      const exists = await pool.query("select 1 from documents where org_id = $1 and id = $2", [orgId, documentId]);
      throw exists.rowCount === 1 ? new Fenced("journal: fenced, a newer owner claimed the document") : new Error("journal: the
document is gone");
    }
    return undefined;
  } catch (err) {
    // Two unique keys, told apart by NAME: 23505 alone cannot say whether this is a resend or a rival writer.
    if (err instanceof Error && "constraint" in err && err.constraint === "op_journal_op") {
      const original = await one(JournalRow, `select ${JOURNAL_COLUMNS} from op_journal where document_id = $1 and actor_id = $2 and
op_id = $3`, [documentId, actor.id, opId]);
      if (original) return original;
    }
    throw err;
  }
},
```

Three TypeScript things in that `append`. `insert ... select ... from documents where org_id = $1 and id = $2` is lesson 1's tenancy rule applied to the journal: the row is written *from* the document's row, so a document of another org (or one deleted meanwhile) takes no op; `rowCount !== 1` is the tell. **"constraint" in err** is the `in` operator used as a type guard: `err` is `unknown` in a `catch`, `instanceof Error` narrows it to `Error`, and `"constraint" in err` narrows it further to `Error & Record<"constraint", unknown>`, so the comparison compiles without a cast. Java would catch `SQLException` and read `getServerErrorMessage().getConstraint()`; `psycopg` gives `err.diag.constraint_name`. **The row type is a Zod schema with a `.transform`**: `seq` is a `bigint` in Postgres and the driver hands it over as a *string* (the same finding lesson 3 hit for usage counts), so `JournalRow` converts once, at the edge, and the rest of the process sees a number.

FROM packages/db/src/index.ts

```
// seq is a bigint: a string from the driver, a number from here on.
const JournalRow = z
  .object({ seq: z.string().regex(/^d+$/).transform(Number), op_id: z.string(), actor_kind: z.string(), actor_id: z.string(), run_id:
z.string().nullable(), op: z.unknown() })
  .transform((r): SequencedOp => SequencedOp.parse({ seq: r.seq, opId: r.op_id, actor: { kind: r.actor_kind, id: r.actor_id, ...(r.run_id
=== null ? {} : { runId: r.run_id }) }, op: r.op }));
```

Now the room's side. The `Journal` type is what the room needs and nothing about Postgres: four functions, injected. Without one (tests, the simulator) nothing is kept, and the room behaves as it did in lesson 2.

FROM apps/sync/src/room.ts

```
/**
 * Where accepted ops are made durable (the op journal, E6.1a). The room's memory of what it applied is
 * bounded and dies with it; the journal's is neither, so with one the room can answer ANY resend.
 */
export type Journal = {
  /** Resolves once the op is durable: undefined, or the op this sender's opId ALREADY became. Rejects: not durable. */
  append(op: SequencedOp): Promise<SequencedOp | undefined>;
  /** What this sender's opId became, if it was ever journaled. */
  find(actorId: string, opId: string): Promise<SequencedOp | undefined>;
  /** Was this node id ever added (keystone 4: a removed id is never added again)? */
  everAdded(nodeId: string): Promise<boolean>;
  /** Every op journaled after 'seq', in order: how a room that went read-only catches up before it writes again. */
  since(seq: number): Promise<SequencedOp[]>;
};
```

FROM apps/sync/src/room.ts

```
/**
 * Makes an op durable BEFORE anyone hears of it. If the append rejects, the op is not applied, nobody
 * receives it, and its sequence number is not used up. Without one nothing is kept (tests, the simulator).
 */
journal?: Journal;
```

FROM apps/sync/src/room.ts

```
// 3. Durable first, then applied, then announced. The number is only TAKEN once the op is durable,
// so a failed write leaves no gap. The ACTOR comes from the verified session (SPEC §2.3).
const sequenced: SequencedOp = { seq: seq + 1, opId, actor: peer.actor, op };
let original;
try {
  original = await journal?.append(sequenced);
} catch {
  refuse("unavailable");
  return;
}
// A resend the room had forgotten, with a baseSeq that got it past the check above (a buggy or lying
// client). The journal's unique key caught it: the original answer, and nothing is applied twice.
if (original) {
  peer.send({ type: "op", ...original });
  return;
}
accept(sequenced);
}
```

FROM apps/sync/src/room.ts

```
/** An op the journal holds and the room does not yet: applied, remembered and announced like a fresh one. */
function accept(sequenced: SequencedOp): void {
  applyOpInto(doc, sequenced.op);
  seq = sequenced.seq;
  nodeCount = sequenced.op.type === "add_node" ? nodeCount + 1 : sequenced.op.type === "remove_node" ? Object.keys(doc.nodes).length :
nodeCount;
  remember(`${sequenced.actor.id}:${sequenced.opId}`, sequenced);
  broadcast({ type: "op", ...sequenced }); // the sender's copy is its acknowledgement
  onAccepted?.(seq);
}
```

Read step 3 slowly, because the whole epic is in its order. `sequenced` is built with `seq + 1`, but `seq` itself is not moved: the number is only *taken* inside `accept`, after the append resolved. A failed write therefore leaves no gap, and the next op gets the same number.

`journal?.append(sequenced)` is optional chaining on a *call*: with no journal it evaluates to `undefined` and the `await` is a no-op. `let original;` with no type and no initialiser is not `any`: TypeScript infers its type from the assignments it can see (an "evolving" `let`), so after the `try` it is `SequencedOp | undefined`. And `accept` is the one function that applies, numbers, remembers and announces, so that `recover()` in section 3 can do exactly what a fresh op does, to a row it found in the journal.

The append happens *inside* the room's one-at-a-time queue. That is what makes the journal's order the broadcast order under concurrent submits: op 2 is not even validated until op 1's append has resolved. It costs one database round trip per op on the critical path (about a millisecond on the same machine, tens over a network), which is the price of the promise; batching appends is the named upgrade, and it would move the acknowledgement, not remove the rule.

FROM apps/sync/src/room.test.ts

```
test("journal: every accepted op is journaled BEFORE it is broadcast, in the order it is broadcast, under concurrent submits", async () => {
  const journal = memoryJournal();
  const order: string[] = [];
  const room = createRoom({ doc: emptyDoc(), manifest, journal: { ...journal, append: async (op) => { await Promise.resolve();
order.push(`journal ${String(op.seq)}`); return journal.append(op); } } });
  const watcher: Peer = { actor: { kind: "user", id: "w" }, session: { userId: "w", orgId: "org", expiresAt: 0 }, mayEdit: true, send: (m)
=> { if (m.type === "op") order.push(`broadcast ${String(m.seq)}`); } };
  const [a, b] = [peer("a"), peer("b")];
  room.join(a); room.join(b); room.join(watcher);
  await Promise.all(Array.from({ length: 20 }, (_, i) => room.submit(i % 2 ? a : b, clientOp(add(`n${String(i)}`)))));
  expect(order).toEqual(Array.from({ length: 20 }, (_, i) => [ `journal ${String(i + 1)}`, `broadcast ${String(i + 1)}` ]).flat());
  expect(journal.rows.map((r) => r.seq)).toEqual(ops(a).map((m) => m.seq));
});
```

FROM packages/db/src/journal.int.test.ts

```
test("a sender's opId again is answered with the ORIGINAL row; a taken seq is refused by name", async () => {
  const store = t.db.documentStore();
  const doc = await aDocument();
  const first = row(1, add("n1"));
  expect(await store.append(doc.orgId, doc.id, first)).toBeUndefined();
  // The same sender and opId under a new number: a resend. Nothing is written, the original comes back.
  expect(await store.append(doc.orgId, doc.id, { ...first, seq: 2 })).toEqual(first);
  // Another sender may use the same opId: every broadcast shows it to everyone, so it is not a resend.
  expect(await store.append(doc.orgId, doc.id, { ...row(2, add("n2"), "u2"), opId: first.opId })).toBeUndefined();
  // A taken number is a rival writer, not a resend.
  await expect(store.append(doc.orgId, doc.id, row(2, add("n3")))).rejects.toMatchObject({ code: "23505", constraint: "op_journal_seq" });
});
```

<pre> expect(await store.since(doc.orgId, doc.id, 0)).toEqual([first, expect.objectContaining({ seq: 2, actor: { kind: "agent", id: "u2", runId: "run-1" } })]); expect(await store.find(doc.orgId, doc.id, "u1", first.opId)).toEqual(first); expect(await store.find(doc.orgId, doc.id, "u3", first.opId)).toBeUndefined(); }); </pre>		
TYPESCRIPT (THIS REPO)	JAVA	PYTHON
"constraint" in err after err instanceof Error: the in operator narrows	catch (SQLException e) { e.getServerErrorMessage().getConstraint() }	except errors.UniqueViolation as e: e.diag.constraint_name
let original; typed by its later assignments	no equivalent; declare the type	no equivalent; a plain name
journal?.append(op) : optional chaining on a call	Optional.map(j -> j.append(op))	journal.append(op) if journal else None
a Zod row schema with .transform to convert bigint text once	a RowMapper that calls getLong	a typecaster on the cursor, or int() in the mapper
insert ... select from documents where org_id = \$1 : the tenant check in the write	the same SQL; JPA cannot express it	the same SQL

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

E2.3's known limit: a client that resent its own forgotten op with a forged baseSeq got a second seq.

Lesson 2 named it and could not fix it: the room's memory of applied ops is bounded, and once an op fell out of it a resend looked like a new op. The room's stale guard (baseSeq < forgottenUpTo) caught honest clients, but a client claiming to have seen everything walked past it. The journal's unique key is the fix, and it is free: the append that would have created the duplicate breaks op_journal_op , the original row comes back, and the room answers with it and applies nothing. The test pushes an op out of a two-entry memory and resends it with baseSeq = room.seq .

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A key on (document, opId) would have refused legitimate ops.

The obvious dedupe key is the opId. But every broadcast shows every opId to every peer, so a peer that saw another's opId could send it as its own, and, more to the point, a bare-opId key would treat that as a resend and answer with the other peer's op. Lesson 2's memory was already keyed by sender for this reason; the journal's constraint is keyed the same way, and the database test proves that two senders may use one opId and both are written.

The principle: an acknowledgement is a promise about the future ("this will still be true after a crash"), so it may only be given by the thing that will still be there after the crash. The room gives it after the database has spoken, never before. Section 8's first row is what happens when the order is reversed, and drill 2 makes you find it.

2 · Exactly once, again: the resend, the forgotten op, and the id that never returns

Lesson 2 already had exactly-once over a lying connection: a client keeps its opIds, resends what was not answered, and the room answers a resend from memory. Epic 6 removes the word "memory" from that sentence. With a journal, every question the room used to answer from a bounded map can be answered from a table, including after a restart.

FROM apps/sync/src/room.ts

```

// What the room can still vouch for: "sender:opId" -> what it became. Insertion-ordered, so the
// oldest is first. Keyed by SENDER too: every broadcast shows every opId to every peer, and a peer
// that replayed someone else's opId must not get their answer while its own op is thrown away.
// With a journal this is only a cache: what fell out of it (or died with the last room) is asked of the journal.
const remembered = new Map<string, { op: SequencedOp; bytes: number }>();
// Ops that changed nothing, so that a resend gets the same answer instead of a second look at a
// document that has moved on. Their OWN small memory: they cost their sender almost nothing, and in
// the map above a flood of them would push out real ops and turn honest resends into "stale".
// ponytail: a no-op that falls out of here is judged afresh if it is resent; the worst case is one
// old value written late. The journal cannot help (a no-op has no row): keep this.

```

FROM apps/sync/src/room.ts

```
// 1. Dedupe FIRST. A client that never saw its acknowledgement sends the op again and must get
// the original answer; validating first would turn an ordinary retry into "duplicate_node".
const key = `${peer.actor.id}:${opId}`;
const before = remembered.get(key);
if (before) {
  peer.send({ type: "op", ...before.op });
  return;
}
if (rememberedNoOps.has(key)) {
  peer.send({ type: "ack", opId });
  return;
}
// An op written before the oldest thing the room remembers MAY already have been applied. The
// journal can tell: found, it gets its original answer; not found, it never was, and is judged now.
// Without a journal the room cannot tell, and applying it twice would move a node twice: resync.
// A client that forged a high baseSeq (buggy or lying) skips that check, so every refusal below,
// and the no-op ack, asks the journal first too: a resend gets its first answer, whatever it claims to
// have seen (noon-mo3.1.1). Asked at most once per op, and only on those rare paths.
let asked = false;
/** True when the op has been answered: with its original row, or "unavailable". */
const answeredFromJournal = async (): Promise<boolean> => {
  if (!journal || asked) return false;
  asked = true;
  let original;
  try {
    original = await journal.find(peer.actor.id, opId);
  } catch {
    refuse("unavailable");
    return true;
  }
  if (original) peer.send({ type: "op", ...original });
  return original !== undefined;
};
if (baseSeq < forgottenUpTo) {
  if (!journal) {
    refuse("stale");
    return;
  }
  if (await answeredFromJournal()) return;
}
```

The dedupe is a two-level cache. The `Map` answers most resends in memory. An op whose `baseSeq` is behind everything the room remembers used to be refused as `stale` ("I cannot tell whether I applied this"); now the journal is asked with `find`: found, the original answer; not found, the op is judged now like any other. The `stale` path is kept for a room without a journal, so the simulator and lesson 2's tests still hold. Note the shape of the error handling: a journal that cannot be *read* refuses the op as `unavailable`, the same reason as a failed write, because in both cases the room does not know and must not guess.

FROM apps/sync/src/room.ts

```
// Keystone 4: validate() only sees the document as it is; an id that was removed is in the journal.
// ponytail: one indexed read per add (adds are rare next to drags); without a journal, unchecked.
if (op.type === "add_node" && journal) {
  let reused;
  try {
    reused = await journal.everAdded(op.nodeId);
  } catch {
    refuse("unavailable");
    return;
  }
  if (reused) {
    if (!(await answeredFromJournal())) refuse("duplicate_node");
    return;
  }
}
```

Keystone 4 says a node id that was removed is never added again, or an edit meant for the dead node lands on the new one. `validate` only sees the document as it is; it cannot know what was removed. Lesson 5's git peer worked around it with `git log -p` for pushed pages. Now the room asks the journal for every actor: one indexed read per add, through a partial index on `(document_id, op ->> 'nodeId')` where `op ->> 'type' = 'add_node'`. The query is spelled exactly as the index is, because Postgres uses a partial index only when the query's predicate implies the index's.

The last piece is *opening*. A room reloaded from a saved document knows only that document's `seq`. If the process died after journaling ops the save never saw, the room would number from a stale `seq` and every append would break `op_journal_seq`. So opening replays `since(seq)` before the room exists:

FROM apps/sync/src/server.ts

```
// The contract checked each node's shape. Whether they form a TREE is checkDoc's job, and a room
// must never open on top of a corrupt document: every later op would build on the damage.
if (checkDoc(doc).length > 0) return { closeCode: CLOSE.documentCorrupt };
try {
  for (const row of await store.since(orgId, documentId, seq)) {
    applyOpInto(doc, row.op);
    seq = row.seq;
  }
} catch {
  return { closeCode: CLOSE.unavailable };
}
```

FROM apps/sync/src/room.test.ts

```
test("journal: a client that resends its own forgotten op with a FORGED high baseSeq gets the original seq, not a second one (E2.3 known limit)", async () => {
  const journal = memoryJournal();
  const room = createRoom({ doc: emptyDoc(), manifest, journal, limits: { rememberedOps: 2 } });
  const liar = peer("liar");
  room.join(liar);
  await room.submit(liar, clientOp(add("n1")));
  const setOnce = clientOp(setGap("n1", 8), 1);
  await room.submit(liar, setOnce); // seq 2
  for (let i = 0; i < 3; i++) await room.submit(liar, clientOp(setGap("n1", 10 + i), 2 + i)); // pushes seq 2 out of the room's memory
  await room.submit(liar, { ...setOnce, baseSeq: room.seq }); // "I have seen everything": the stale guard is passed
  expect(ops(liar).at(-1)).toMatchObject({ seq: 2, opId: setOnce.opId });
  expect(room.doc.nodes["n1"]?.props["gap"]).toBe(12); // the old value was NOT written again
  expect(journal.rows.filter((r) => r.opId === setOnce.opId)).toHaveLength(1);
});
```

FROM apps/sync/src/room.test.ts

```
test("journal: a node id that was added and removed is never added again, even after a restart (keystone 4)", async () => {
  const journal = memoryJournal();
  const first = createRoom({ doc: emptyDoc(), manifest, journal });
  const a = peer("a");
  first.join(a);
  await first.submit(a, clientOp(add("gone")));
  await first.submit(a, clientOp({ type: "remove_node", nodeId: "gone" }, 1));
  const again = createRoom({ doc: structuredClone(first.doc), seq: first.seq, manifest, journal });
  const git = peer("git-peer", "git");
  again.join(git);
  const readd = clientOp(add("gone"), 2);
  await again.submit(git, readd);
  expect(rejects(git)).toEqual([{ type: "rejected", opId: readd.opId, reason: "duplicate_node" }]);
  expect(again.doc.nodes["gone"]).toBeUndefined();
});
```

FROM apps/sync/src/journal.int.test.ts

```
test("a resent op gets its original seq after a restart, even when the process died before saving the document", async () => {
  const doc = await aDocument();
  const userId = randomUUID();
  let server = await start();
  const a = await connect(server.url, doc.id, userId, {}, doc.orgId);
  await a.next("welcome");
  const ops = [add("outer"), add("inner", "outer"), { type: "set_prop", nodeId: "outer", key: "gap", value: 24 } satisfies Op];
  const opIds: string[] = [];
  for (const op of ops) {
    opIds.push(a.send(op));
    await a.next("op", (m) => m.opId === opIds.at(-1)); // one at a time, so each resend below has a known baseSeq
  }
  const expected = ops.reduce<Doc>(applyOp, emptyDoc());
  await server.close();
  await a.closed;
  // No snapshot store here: only the journal knows about seq 1..3, as after a crash before any snapshot.

  server = await start();
  try {
    const back = await connect(server.url, doc.id, userId, {}, doc.orgId);
    // The room replayed the journal: nothing acknowledged is lost, and numbering continues from 3.
    expect(await back.next("welcome")).toMatchObject({ doc: expected, seq: 3 });
    // The ack of the set_prop never arrived. Resent as a real client would: with the baseSeq it had then.
    back.sendRaw({ type: "op", opId: opIds[2], baseSeq: 2, op: ops[2] });
    expect(await back.next("op", (m) => m.opId === opIds[2])).toMatchObject({ seq: 3, actor: { kind: "user", id: userId } });
    // A buggy or lying client resends the first op claiming to have seen everything: the unique key answers.
    back.sendRaw({ type: "op", opId: opIds[0], baseSeq: 3, op: ops[0] });
  }
}
```

```

expect(await back.next("op", (m) => m.opId === opIds[0])).toMatchObject({ seq: 1 });
// And "inner" was removed... then re-added under the same id: refused, the journal remembers it.
const removed = back.send({ type: "remove_node", nodeId: "inner" });
expect((await back.next("op", (m) => m.opId === removed)).seq).toBe(4);
const readd = back.send(add("inner", "outer"));
expect(await back.next("rejected", (m) => m.opId === readd)).toMatchObject({ reason: "duplicate_node" });

const fresh = back.send(add("after-restart"));
expect((await back.next("op", (m) => m.opId === fresh)).seq).toBe(5);
const journal = (await t.rawQuery("select seq, op_id from op_journal where document_id = $1 order by seq", [doc.id])) as { rows: {
  seq: string; op_id: string }[] };
expect(journal.rows.map((r) => r.seq)).toEqual(["1", "2", "3", "4", "5"]); // no gap, and no op twice
expect(new Set(journal.rows.map((r) => r.op_id)).size).toBe(5);
back.close();
await back.closed;
} finally {
  await server.close();
}
});

```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

a `Map` in front of a table: the memory answers, the table is the truth

Caffeine in front of JPA; the same discipline about which one is the truth

`functools.lru_cache` in front of a query

create index ... where op -> 'type' = 'add_node' : a partial index on a JSON path

the same SQL; Hibernate cannot express it

the same; Django `Index(condition=...)`

for (const row of await store.since(...))
applyOpInto(doc, row.op)

a `for over a List` from a repository

a `for over cursor.fetchall()`

`refuse("unavailable")` on a read failure too

the same rule: unknown is not "no"

the same

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

E5.3b's gap: an id removed on the canvas that never reached git could be re-added by a push.

Lesson 5 computed `earlierIds` from the repository's history, so an id that lived and died on the canvas *between* two generations of the file was unknown to the git peer, and a push could bring it back. The room now refuses it for every actor from the journal, so the git peer's push is refused op by op (`duplicate_node`). What it does *not* yet do is turn that refusal into a whole-page conflict banner; folding the journal's ids into `earlierIds` is the named follow-up.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

`replica.ts` 's `everSent` rule could now go, and was left.

The peer-client refused to resend an op it had once sent under a seq it no longer held, because a room might apply it twice. With the journal, the room cannot. The rule is now redundant but harmless, and removing it belongs to a peer-client bead, not to a sync one: the commit says so, which is how a codebase avoids two teams reaching for the same file.

What "exactly once" means here, precisely: a client may send an op any number of times; the room applies it at most once; and every sending after the first is answered with the same `seq`. "At most once" comes from the unique key; "at least once" comes from the client's resend; the answer being *the same* comes from storing the original row and returning it. Remove any one of the three and the promise is gone.

3 · The database is gone: read-only, and the recovery that reads before it writes

SPEC §4 says: *Postgres or MinIO down: rooms refuse new ops with a visible read-only status; nothing is silently dropped*. E6.1b is that row, and it turned out to be the most delicate design in the epic, because two things must hold at once: **no acknowledgement while the journal cannot be written** (an acknowledgement is a promise), and **no op lost** (the person's edit is theirs). The resolution is that the room refuses and the *client* holds.

FROM `packages/contracts/src/index.ts`

```

// To EVERY peer when the room's storage fails or comes back (E6.1b, SPEC §4). While read-only the room
// refuses every op as "unavailable" and acknowledges none: a peer holds its edits and sends them again
// once it hears 'readOnly: false'. An older client skips this type and falls back on the refusals.
z.object({ type: z.literal("status"), readOnly: z.boolean() }),
]);

```

FROM apps/sync/src/room.ts

```
// E6.1b (SPEC §4): the journal failed, so nothing more can be made durable. Until recover() has caught up
// with the journal, EVERY op is refused and none is acknowledged, and every peer has been told why.
// Any journal failure counts, a lost reply included: whether that append landed is only known by reading.
let readOnly = false;
```

FROM apps/sync/src/room.ts

```
function storageFailed(): void {
  if (readOnly) return;
  readOnly = true;
  broadcast({ type: "status", readOnly: true }); // to everyone, and BEFORE the refusal that caused it
  onReadOnly?();
}

async function handle(peer: Peer, { opId, baseSeq, op }: ClientOp): Promise<void> {
  const refuse = (reason: Extract<ServerMessage, { type: "rejected" }>["reason"]): void => {
    if (reason === "unavailable") storageFailed();
    peer.send({ type: "rejected", opId, reason });
  };

  if (!peers.has(peer)) return; // dropped while this op waited in the queue
  // F24: asked when the op's TURN comes, not when it arrived: an op queued before a demotion is refused too.
  if (!peer.mayEdit) {
    refuse("forbidden");
    return;
  }
  // Before the dedupe: even an answer from memory is an acknowledgement, and a read-only room gives none.
  if (readOnly) {
    refuse("unavailable");
    return;
  }
}
```

Order matters in `storageFailed`: the `status` goes to every peer *before* the refusal that caused it goes to the sender. So when the sender's client sees `rejected: unavailable` it already knows the room is read-only and treats the refusal as "hold this", not "resync". And read the placement of the `readOnly` check in `handle`: before the dedupe. Even an answer from memory is an acknowledgement, and a read-only room gives none, not for a fresh op, not for a no-op, not for a resend it remembers. The test spells that list out.

FROM apps/sync/src/room.ts

```
/**
 * Leaves read-only if the journal answers again. It first REPLAYS whatever the journal holds past the
 * room's seq (an append whose reply was lost, or a rival's row) and announces it, then tells every peer
 * "writable": a held op that did land is thus confirmed before its sender sends it again. Takes its turn
 * in the op queue, so no op is judged halfway through. True = writable. Never rejects.
 */
recover(): Promise<boolean> {
  const done = tail.then(async () => {
    if (!readOnly || !journal) return true;
    let missed;
    try {
      missed = await journal.since(seq);
    } catch {
      return false;
    }
    for (const row of missed) accept(row);
    readOnly = false;
    broadcast({ type: "status", readOnly: false });
    return true;
  });
  tail = done.then(() => undefined, () => undefined);
  return done.catch(() => false);
},
```

`recover()` is the design decision of the bead. The obvious version is a health probe: ping the database, and if it answers, say "writable". It was rejected for two failures it would cause. First, an append can *land* and still throw (the connection dropped after the commit): the room refused the op, the client holds it, and if the room goes writable without reading, the client's resend hits `op_journal_op`, gets the original row, and the room applies it, which is fine, but every other peer never heard the op, because the original broadcast never happened. Second, a rival row at the room's next seq (section 1's other constraint) would keep the room refusing until a restart. So `recover()` first *replays* `since(seq)` through `accept`, the same function a fresh op goes through, and only then says writable. It takes its turn in the queue, so no op is judged halfway through a replay.

Two idioms. `tail = done.then(() => undefined, () => undefined)` puts the recovery on the queue and makes sure a failure in it cannot jam the queue for every later op (`then` with two arguments handles both outcomes). `return done.catch(() => false)` gives the caller a boolean that never rejects. The room has no clock, so it cannot retry by itself; `onReadOnly` tells the server to start.

FROM `apps/sync/src/server.ts`

```
// A timed-out append may still land later: the room treats it as failed, and recover() replays it if it did.
// A fenced one never lands: the room is dropped, not healed (a newer owner is writing).
let fenced = false;
// An edit reaches the other browsers only after these answer: a slow one is logged, with which it was.
const timed = reportSlow(SLOW_JOURNAL_MS, (what, ms) => { log(documentId, `journal ${what} took ${String(ms)} ms`, "warn"); });
const journal = {
  append: watchFence((op: SequencedOp) => timed("append", bounded(store.append(orgId, documentId, op, fence?.claim))), () => {
    fenced = true;
    fence?.onFenced();
  }),
  find: (actorId: string, opId: string) => timed("find", bounded(store.find(orgId, documentId, actorId, opId))),
  everAdded: (nodeId: string) => timed("everAdded", bounded(store.everAdded(orgId, documentId, nodeId))),
  since: (after: number) => timed("since", bounded(store.since(orgId, documentId, after))),
};
```

FROM `apps/sync/src/server.ts`

```
function bounded<T>(work: Promise<T>): Promise<T> {
  let timer: NodeJS.Timeout | undefined;
  const late = new Promise<never>((_, reject) => { timer = setTimeout(() => { reject(new Error("journal timed out")); },
journalTimeoutMs); });
  return Promise.race([work, late]).finally(() => { clearTimeout(timer); });
}

/**
 * The room went read-only (E6.1b): ask it to recover every 'recoverMs' until the journal answers.
 * ponytail: a read-only room that was dropped meanwhile keeps asking too, one cheap read a tick, until
 * the database is back; ceiling: one timer per such room, for as long as the outage lasts.
 */
function heal(room: Room): void {
  void (async () => {
    while (room.readOnly && !closing) {
      await new Promise((resolve) => setTimeout(resolve, recoverMs).unref());
      await room.recover();
    }
  })();
}
```

`bounded` is the timeout: `Promise.race` between the work and a timer that rejects, with `finally` clearing the timer so a fast answer does not leave one ticking. A *paused* database (the second half of the chaos check) never refuses a query; it hangs, and without this the room would wait for ever with every peer's ops queued behind it. Note what the timeout means: "treat as failed", not "it failed": the append may still land later, which is exactly why `recover()` reads before it writes. `heal` is an async IIFE (`void (async () => { ... })()`) that loops with a `setTimeout` promise whose timer is `unref()` 'd so it cannot keep the process alive at shutdown.

The client's half. peer-client learns one flag from the wire and does four things with it.

FROM `packages/peer-client/src/peer.ts`

```
// E6.1b: the room cannot make edits durable. What we sent is held, new edits are refused, until it says otherwise.
let readOnly = false;
let pauseTimer: ReturnType<typeof setTimeout> | undefined;
const endPause = (): void => {
  clearTimeout(pauseTimer);
  pauseTimer = undefined;
  lastHeard = Date.now(); // the silence during a pause was ours, not the server's
};
```

FROM `packages/peer-client/src/peer.ts`

```
/** Writes whatever the replica says may go out now. Not live, or told to slow down: it all stays pending. */
function flush(): void {
  if (status !== "live" || pauseTimer !== undefined || readOnly) return;
  for (const op of replica.takeSendable()) socket?.send(JSON.stringify({ type: "op", ...op } satisfies ClientMessage));
}
```

FROM `packages/peer-client/src/peer.ts`

```
if (message.type === "status") {
  readOnly = message.readOnly;
  onChange?();
  flush(); // writable again: the held edits go out, in order
  return;
}
```

FROM packages/peer-client/src/peer.ts

```
// A refusal from a room that said it is read-only: held, not a reason to reconnect. It says when it is writable.
const held = readOnly && message.type === "rejected" && message.reason === "unavailable";
if (effects.resync && !held) { resync(); return; }
```

FROM packages/peer-client/src/peer.ts

```
// Said at once, not queued behind a room that cannot save: a person sees why, a program (the AI, the git peer) stops or waits.
if (readOnly) return { ok: false, reason: "read_only" };
```

Held ops are *not sent again and not resynced*: `flush` sends nothing while read-only, and a refusal with `unavailable` under a read-only room does not trigger the resync a bare `unavailable` would. New edits are refused at once with `read_only`, so a person sees why and a program stops. And "writable" simply calls `flush`: the held ops go out in their original order with their original opIds, so if any of them landed during the outage, the room answers from the journal. The watchdog is turned off while read-only, a named ponytail: a connection that dies half-open then is noticed by the next edit or by presence going quiet.

Every kind of peer hears it through the one contract. The canvas shows the status and an alert and disables the library. The AI's tool call gets `read_only` with a hint to stop. The git peer throws `WaitAgain`, and its event goes back to `pending` instead of `failed`: a push is never lost to an outage, because lesson 5's reconcile would never re-record it.

FROM apps/web/src/Canvas.tsx

```
{/* E6.1b: the room's storage is down. An alert, so a screen reader says it the moment it happens. */}
{readOnly && <p role="alert" className="read-only">Read-only: the server cannot save edits right now. Edits you already made are kept and will be saved when it can; new edits are paused.</p>}
```

FROM apps/web/src/Canvas.tsx

```
<LibraryPanel doc={doc} selected={node.id} labels={labels} isContainer={isContainer} disabled={readOnly} onCarry={setCarry}
onAdd={addAt} />
```

FROM apps/worker/src/tools.ts

```
read_only: "The document is read-only: the server cannot save edits right now. Stop and report that the edit could not be made.",
```

FROM apps/worker/src/push.ts

```
// E6.1b: the room cannot save. Not a conflict (the page is fine) and not a failure (the push would be
// lost): the event waits. Ops of this page already sent die with the peer below, unapplied.
if (!submitted.ok && submitted.reason === "read_only") throw new WaitAgain("document_read_only");
```

FROM apps/worker/src/git.ts

```
if (err instanceof WaitAgain) {
  // ponytail: the whole event waits, pages already applied included (applying again is safe: push-ops takes
  // the commit's journaled adds as its own, noon-91u). Ceiling: one retry per poll tick while the room stays read-only.
  await store.finish(event, "pending");
  log(`${event.ref} ${event.after} waits: ${err.message}`);
  return false;
}
```

FROM apps/sync/src/room.test.ts

```
test("recover: an append that LANDED although its reply was lost is replayed to everyone with its seq, so the resend is answered, not applied twice", async () => {
  const { journal, state, rows } = flakyJournal();
  const room = createRoom({ doc: emptyDoc(), manifest, journal });
  const [a, b] = [peer("a"), peer("b")];
  room.join(a); room.join(b);
  state.committedAnyway = true;
  const op = clientOp(add("n1"));
  await room.submit(a, op);
  expect(rejects(a).map((r) => r.reason)).toEqual(["unavailable"]);
  expect(ops(b)).toEqual([]); // nobody heard of it: it was not KNOWN to be durable
  expect(await room.recover()).toBe(true);
  // The journal is the truth: the op is in it, so the room applies it and everyone hears it, BEFORE "writable".
  expect(b.inbox.slice(-2)).toMatchObject([{ type: "op", seq: 1, opId: op.opId, actor: { id: "a" } }, { type: "status", readOnly: false }]);
  expect(room.doc.nodes["n1"]).toBeDefined();
  await room.submit(a, op); // the sender's held copy comes back
  expect(ops(a).filter((m) => m.opId === op.opId).map((m) => m.seq)).toEqual([1, 1]);
});
```

```

expect(rows).toHaveLength(1);
const next = clientOp(add("n2"), 1);
await room.submit(a, next);
expect(ops(b).at(-1)).toMatchObject({ seq: 2, opId: next.opId });
});

```

FROM `packages/peer-client/src/peer.test.ts`

```

test("read-only: ops the room refused are HELD (no reconnect, nothing lost), new edits are refused 'read_only', and 'writable' sends the held ones again, in order, with the same opIds", async () => {
  let changes = 0;
  const { net, peer, socket, frames } = await readOnlyPeer({ onChange: () => { changes++; } });
  peer.submit(add("a"));
  peer.submit(add("b"));
  const [a, b] = frames();
  const before = changes;
  socket?.say({ type: "status", readOnly: true });
  expect(peer.readOnly).toBe(true);
  expect(changes).toBeGreaterThan(before); // a UI hears of it
  socket?.say({ type: "rejected", opId: a?.opId, reason: "unavailable" });
  socket?.say({ type: "rejected", opId: b?.opId, reason: "unavailable" });
  expect(peer.submit(add("c"))).toEqual({ ok: false, reason: "read_only" });
  await sleep(50);
  expect(net.sockets).toHaveLength(1); // no resync: the room said why, and will say when
  expect(peer.pendingCount).toBe(2);
  expect(peer.doc.nodes["a"]).toBeDefined(); // the user still sees what they made
  expect(frames()).toHaveLength(2); // nothing is sent into a read-only room
  socket?.say({ type: "status", readOnly: false });
  expect(peer.readOnly).toBe(false);
  expect(frames().slice(2).map((f) => f.opId)).toEqual([a?.opId, b?.opId]);
  peer.close();
});

```

FROM `apps/sync/src/read-only.int.test.ts`

```

// integration:read-only-reaches-every-peer
test("storage down: every peer (person, AI, git) shows read-only, no op is acknowledged; storage back: all writable, the held op lands once", async () => {
  const { store, rows, state } = switchableStore();
  const server = await startSyncServer({ port: 0, secrets: [TEST_SECRET], store, recoverMs: 50 });
  try {
    const documentId = randomUUID();
    const person = peerAs(server.url, documentId, { kind: "user" });
    const ai = peerAs(server.url, documentId, { kind: "agent", runId: randomUUID() });
    const git = peerAs(server.url, documentId, { kind: "git", runId: "b.repeat(40) });
    const all = [person, ai, git];
    await until(() => all.every((p) => p.status === "live"), "all three live");
    const first = person.submit(add("before"));
    expect(first.ok && (await first.settled)).toMatchObject({ ok: true, seq: 1 });

    state.down = true;
    const held = person.submit(add("during"));
    await until(() => all.every((p) => p.readOnly), "every peer read-only");
    expect(ai.submit(add("ai-during"))).toEqual({ ok: false, reason: "read_only" });
    expect(git.submit(add("git-during"))).toEqual({ ok: false, reason: "read_only" });
    await new Promise((resolve) => setTimeout(resolve, 200)); // several recover() attempts, all refused
    expect(person.pendingCount).toBe(1); // held, not lost
    expect(all.every((p) => p.status === "live" && p.readOnly)).toBe(true); // no reconnect storm, still read-only
    expect(rows.map((r) => r.seq)).toEqual([1]);

    state.down = false;
    await until(() => all.every((p) => !p.readOnly), "every peer writable again", 3000);
    expect(held.ok && (await held.settled)).toMatchObject({ ok: true, seq: 2 });
    await until(() => all.every((p) => p.seq === 2), "every peer at seq 2");
    expect(rows.map((r) => [r.seq, r.op.type === "add_node" ? r.op.nodeId : ""])).toEqual([[1, "before"], [2, "during"]]);
    for (const p of all) expect(Object.keys(p.confirmed.nodes).sort()).toEqual(["before", "during", ROOT_ID].sort());
    for (const p of all) p.close();
  } finally {
    await server.close();
  }
});

```

Typescript (this repo)	Java	Python
<code>Promise.race([work, late]).finally(() => clearTimeout(timer))</code>	<code>CompletableFuture.orTimeout</code> ; the work is not cancelled either	<code>asyncio.wait_for</code> , which DOES cancel the task: a real difference
<code>void (async () => { while (...) { await sleep(); await room.recover(); } })()</code>	a <code>ScheduledExecutorService</code> task that reschedules itself	<code>asyncio.create_task</code> of a loop
<code>setTimeout(resolve, ms).unref()</code>	a daemon thread	no equivalent needed; the loop ends with the process
<code>done.then(() => undefined, () => undefined):</code> both outcomes handled	<code>handle((v, e) -> null)</code>	<code>try/except</code> around the <code>await</code>
<code>class WaitAgain extends Error {}</code> as a control-flow signal	a checked exception type	a custom exception class

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A bare health probe would have gone writable with the room behind the journal.

The first design was "ping Postgres every second; when it answers, broadcast writable". Written out against the tests, it failed the lost-reply case: the append landed, the room never applied it, and the next append from anyone hit `op_journal_seq` and put the room straight back into read-only, for ever. Reading `since(seq)` first is not an optimisation of the probe; it is the only correct probe, because "the database answers" and "the room agrees with the database" are different facts.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The refusal of a held op was, to lesson 2's client, a reason to resync.

`unavailable` in lesson 2 meant "the persist step failed, resync and try again", and a resync throws away the client's view and asks for a welcome. Under a read-only room that would be a reconnect storm: every held op refused, every refusal a resync, every welcome saying read-only again. The status message is what breaks the loop: a refusal under a room that *said* it is read-only is a hold. The peer test counts sockets: one.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A git peer whose event failed during an outage would have lost the push for good.

Lesson 5's `git_events` keys on `(ref, after_sha)`, so an event that ends `failed` is never recorded again by the reconcile. A push during a database outage must therefore not end at all: `WaitAgain` puts the event back to `pending`, and the next poll tick tries the whole event again. The ceiling is named (pages already applied are diffed again against the room as it is then, which is a no-op) and the alternative (a per-page cursor on the event) is the upgrade.

The distributed-systems shape: a single writer that stops writing the moment it cannot confirm a write; clients that keep their intent and re-offer it under the same identity; and a recovery that reconciles the writer with the log before it accepts anything new. Every piece is visible on a screen ("read-only", "waiting to save 1"), because a failure nobody can see is a failure nobody can act on.

4 · Snapshots: a cache of the log, written once, named after it exists

With a journal, a document is fully defined by its rows: replay them from an empty page and you have it. That is correct and slow, and it grows without bound. F19 caps the cost: a room writes its whole document to MinIO every 500 accepted ops, every 30 seconds while anyone is connected, and when its last peer leaves or the server stops; opening a document loads the newest snapshot and replays only the rows after it. The journal stays the truth. A snapshot that fails to write loses nothing; it only leaves more rows for the next open.

FROM `packages/db/migrations/0013_snapshot_seq.sql`

```
-- E6.2 (F19): the seq of the newest snapshot in MinIO. S3 cannot list in reverse (learning-tests/minio), so
-- the latest one is found here, not by listing. Written only AFTER the object is stored, so it never names a
-- missing snapshot, and it only moves forward. `content`/'seq' (F8's idle save) are no longer written: they
-- stay as what a document saved before E6.2 opens from until its first snapshot.
alter table documents add column snapshot_seq bigint not null default 0 check (snapshot_seq >= 0);
```

FROM apps/sync/src/snapshots.ts

```
/** Where the snapshot of a document at `seq` lives. Zero-padded to 16 digits (every safe integer): S3 lists keys as strings, so unpadded "10" sorts before "9". */
export function snapshotKey(orgId: string, documentId: string, seq: number): string {
  if (!Number.isSafeInteger(seq) || seq < 1) throw new Error(`snapshot: not a seq: ${String(seq)}`);
  return `${orgId}/${documentId}/${String(seq).padStart(16, "0")}.json.gz`;
}

/** Gzipped by hand: the SDK neither compresses nor decompresses (learning-tests/minio). */
export const encodeSnapshot = (doc: Doc): Uint8Array => gzipSync(JSON.stringify(doc));

/** The document, or undefined when the bytes are not a well-formed tree: a room must never open on top of that. */
export function decodeSnapshot(bytes: Uint8Array): Doc | undefined {
  let parsed;
  try {
    parsed = Doc.safeParse(JSON.parse(gunzipSync(bytes).toString("utf8")));
  } catch {
    return undefined;
  }
  return parsed.success && checkDoc(parsed.data).length === 0 ? parsed.data : undefined;
}
```

Two facts from `learning-tests/minio` decide the key and the pointer. S3 lists keys as strings, so "10" sorts before "9": the seq is zero-padded to 16 digits, enough for every safe integer, and `snapshotKey` refuses anything that is not one. And S3 cannot list in reverse (pages are ascending, 1000 keys each, whatever `MaxKeys` asks), so "the newest snapshot" is not found by listing: `documents.snapshot_seq` names it. `gzipSync` from `node:zlib` compresses by hand because the SDK does neither direction; the result is a `Uint8Array`, the web-standard byte array that Node's `Buffer` extends.

FROM learning-tests/minio/test.ts

```
// 5. Assumed IfNoneMatch: '*' -> 412 on existing key. ACTUAL: MinIO
// RELEASE.2025-09-07 supports it and it works correctly: first PutObject
// with IfNoneMatch:'*' on a fresh key succeeds (200), a second
// PutObject with IfNoneMatch:'*' on the SAME existing key fails with HTTP
// 412 and an error whose SDK-parsed `name` is "PreconditionFailed". So the
// "two rooms can't both write the same snapshot key" guarantee holds on
// this MinIO version. (Older MinIO releases, pre ~2024-06, did NOT support
// conditional writes -- pin MinIO to a recent release if this matters.)
// 6. Confirmed. GetObject on a missing key throws an error with
// `err.name === "NoSuchKey"` and `err.$metadata?.statusCode === 404`.
// Cleanest TS detection: `if (err instanceof S3ServiceException && err.name
// === "NoSuchKey")` (or just check `.name`, since NoSuchKey is a stable
// modeled exception name in @aws-sdk/client-s3) rather than string-matching
// the message.
```

FROM apps/sync/src/snapshots.ts

```
export type SnapshotStore = {
  /** Stores the bytes, once: a key that exists already is left as it is (IfNoneMatch). Rejects when not stored. */
  put(orgId: string, documentId: string, seq: number, body: Uint8Array): Promise<void>;
  /** The bytes, or undefined when there is no such object. Rejects when MinIO does not answer. */
  get(orgId: string, documentId: string, seq: number): Promise<Uint8Array | undefined>;
};
```

FROM apps/sync/src/snapshots.ts

```
async put(orgId, documentId, seq, body) {
  try {
    await s3.send(new PutObjectCommand({ Bucket: bucket, Key: snapshotKey(orgId, documentId, seq), Body: body, ContentType: "application/json", ContentEncoding: "gzip", IfNoneMatch: "*" }));
  } catch (err) {
    // 412: this seq of this document is stored already. The same journal prefix gives the same document,
    // so the object there is this one; overwriting it would only open a window where it is half-written.
    if (!named(err, "PreconditionFailed")) throw err;
  }
},
async get(orgId, documentId, seq) {
  try {
    const res = await s3.send(new GetObjectCommand({ Bucket: bucket, Key: snapshotKey(orgId, documentId, seq) }));
    return res.Body ? await res.Body.transformToByteArray() : undefined;
  } catch (err) {
    if (named(err, "NoSuchKey")) return undefined;
    throw err;
  }
},
```

```
};
}
```

`IfNoneMatch`: "*" is a conditional write: store this object only if no object exists at this key. A 412 (`PreconditionFailed`) means "it is there already", and `put` treats that as success, because the same journal prefix gives the same document, so whatever is at that key is this snapshot; overwriting it would only open a window where the object is half-written. `NoSuchKey` on `get` is "there is no such object" and is answered with `undefined`; every other failure propagates, because "MinIO did not answer" must never read as "there is no snapshot". The SDK's errors are told apart by `err.name`, a stable modelled string, through the small `named` helper: the same shape as section 1's constraint names.

FROM `apps/sync/src/snapshots.ts`

```
/**
 * When a room snapshots (SPEC §2.9): every `everyOps` accepted ops, every `everyMs` while peers are connected
 * (a session that is never idle still snapshots), and when asked (the last peer left, the server stops).
 * One write at a time; a trigger during a write is dropped, and the next op or tick catches up.
 */
export function snapshotter({ room, from, cadence, write }: { room: Source; from: number; cadence: SnapshotCadence; write: (seq: number, body: Uint8Array) => Promise<void> }) {
  let saved = from;
  let writing: Promise<unknown> | undefined;

  /** Snapshots the room as it is now, if it moved since the last one. True: stored (or nothing to store). Never rejects. */
  async function take(): Promise<boolean> {
    while (writing) await writing;
    const seq = room.seq;
    if (seq <= saved) return true;
    // Read in the same tick as `seq`: the room edits `doc` in place, and only here is it exactly the document at `seq`.
    const work = write(seq, encodeSnapshot(room.doc)).then(() => { saved = Math.max(saved, seq); return true; }, () => false);
    writing = work.finally(() => { writing = undefined; });
    return work;
  }

  const timer = setInterval(() => { if (room.peerCount > 0 && !writing) void take(); }, cadence.everyMs);
  timer.unref();
  return {
    get saved() { return saved; },
    /** The room accepted an op numbered `seq`. */
    accepted(seq: number): void {
      if (!writing && seq - saved >= cadence.everyOps) void take();
    },
    take,
    stop(): void { clearInterval(timer); },
  };
}
```

The cadence is a pure module fed by the room's `onAccepted` hook and a timer. `while (writing) await writing;` is "wait for any write in flight": a plain loop over a promise field, the same one-at-a-time idea lesson 5 wrote as a promise chain. The comment marked *Read in the same tick as* `seq` is the correctness heart: the room edits `doc` in place, so the document is exactly the document at `seq` only until the next `await`. `encodeSnapshot(room.doc)` is called synchronously, before `write` is awaited. Move it after an `await` and a snapshot named `seq` could hold `seq+3`'s document.

FROM `apps/sync/src/server.ts`

```
const snapshotFrom = Math.max(stored.snapshotSeq, stored.seq);
let snapshot: ReturnType<typeof snapshotter> | undefined;
const room: Room = createRoom({
  doc, seq, manifest, limits: roomLimits, journal, ...(rate ? { rate } : {}),
  onReadOnly: () => { if (!fenced) heal(room); },
  onAccepted: (accepted) => { snapshot?.accepted(accepted); },
});
if (snapshots) {
  const to = snapshots;
  // MinIO first, THEN Postgres: the pointer never names an object that is not there. Either failing
  // loses nothing (the journal holds every op), it only leaves more rows for the next open to replay.
  snapshot = snapshotter({
    room, from: snapshotFrom, cadence,
    write: async (at, body) => {
      try {
        await to.put(orgId, documentId, at, body);
        await store.snapshottd(orgId, documentId, at);
      } catch (err) {
        log(documentId, `snapshot ${String(at)} failed: ${err instanceof Error ? err.message : "unknown"}`);
        throw err;
      }
    },
  });
}
```

```

    return { room, orgId, ...(snapshot ? { snapshot } : {}) };
  }

```

MinIO first, then Postgres. The object is stored, and only then is `snapshot_seq` moved. So the pointer never names a missing object: if the process dies between the two, the object is orphaned (harmless) and the pointer still names the previous snapshot (correct). `snapshotted`'s `where snapshot_seq < $3` makes the pointer monotonic, so a slow write of an older snapshot finishing after a newer one cannot point the document backwards.

FROM `packages/db/src/index.ts`

```

async snapshotted(orgId, documentId, seq) {
  if (!isId(orgId) || !isId(documentId)) return;
  // '< $3': a slow write of an older snapshot must never point the document back at it.
  await pool.query("update documents set snapshot_seq = $3 where org_id = $1 and id = $2 and snapshot_seq < $3", [orgId, documentId, seq]);
},

```

FROM `apps/sync/src/server.ts`

```

// The newest snapshot, or what F8's idle save wrote before E6.2 if that is newer (a document not
// snapshotted since). Then only the journal rows after it (F19).
let doc: Doc;
let seq: number;
if (stored.snapshotSeq > stored.seq) {
  if (!snapshots) return { closeCode: CLOSE.unavailable }; // snapshotted, and this process was started without MinIO
  let bytes;
  try {
    bytes = await snapshots.get(orgId, documentId, stored.snapshotSeq);
  } catch {
    return { closeCode: CLOSE.unavailable }; // MinIO is down: "try again"
  }
  // Postgres names it only after it was stored, so a missing or unreadable one is damage, not an outage.
  const decoded = bytes && decodeSnapshot(bytes);
  if (!decoded) {
    log(documentId, `snapshot ${String(stored.snapshotSeq)} is ${bytes ? "not a well-formed document" : "missing"}`);
    return { closeCode: CLOSE.documentCorrupt };
  }
  doc = decoded;
  seq = stored.snapshotSeq;
} else {
  doc = stored.doc ?? emptyDoc();
  seq = stored.seq;
}

```

Opening picks the newer of two starting points: the snapshot, or what F8's idle save wrote before E6.2 for a document not snapshotted since. Then the same `since(seq)` replay as section 2. The close codes carry the distinction a user needs: MinIO down is 4503 ("try again"); a snapshot that is missing or malformed although Postgres names it is 4500 ("damage"), because Postgres only names an object after it was stored. `decodeSnapshot` runs the bytes through `Doc.safeParse` and `checkDoc`, so a room never opens on top of a tree that is not one.

FROM `apps/sync/src/server.ts`

```

/**
 * The last peer left: snapshot, then forget the room. A failed snapshot is NOT retried: the journal holds
 * every op this room accepted (E6.1a), so the next open replays what the snapshot would have held.
 * This replaced F8's save, which had to retry for ever because the room's memory was the only copy.
 */
function closeRoom(documentId: string, opened: Owned, opening: Promise<Opened>): void {
  const work = (async () => {
    await snapshotNow(opened);
    // Someone may have joined WHILE the snapshot was written: then the room stays.
    if (opened.room.peerCount === 0 && (opened.lost || rooms.get(documentId) === opening)) {
      opened.snapshot?.stop();
      if (rooms.get(documentId) === opening) {
        rooms.delete(documentId);
        peerCounts.delete(documentId);
      }
      await opened.release(); // after the snapshot: the next owner opens from it
    }
  })();
  leaving.add(work);
  void work.finally(() => leaving.delete(work));
}

```

FROM apps/sync/src/server.ts

```
close: async () => {
  closing = true;
  clearInterval(beat);
  clearInterval(sweeper);
  // FIRST snapshot every open room, while its peers are still connected. Terminating the sockets
  // first (under F8's save) lost everything: their 'close' handlers only run on a later tick, so
  // "nothing pending" was true and the database pool was closed.
  const open = await Promise.all([...rooms.values()]);
  await Promise.all(open.map(async (opened) => {
    if (!("room" in opened)) return;
    await snapshotNow(opened);
    opened.snapshot?.stop();
    await opened.release(); // another node may take the room at once, not a ttl later
  }));
```

F8's save is deleted, and with it a retry loop and a setting. It had to retry for ever because the room's memory was the only copy. Now a failed snapshot is *logged and not retried*; the rejected alternative, retrying on last leave, would have kept complexity that the journal made pointless. The shutdown order is the same lesson 2 learned the hard way: snapshot every room *while its peers are still connected*, then terminate the sockets, then close the pool.

FROM apps/sync/src/config.ts

```
// E6.2: where rooms write their snapshots. No defaults: an address or a password is never guessed.
MINIO_URL: z.string({ error: "MINIO_URL is required" }).refine((value) => URL.canParse(value) && /^https?:$/.test(new
URL(value).protocol), "MINIO_URL must be an http(s):// URL"),
MINIO_USER: z.string({ error: "MINIO_USER is required" }).min(1, "MINIO_USER is required"),
MINIO_PASSWORD: z.string({ error: "MINIO_PASSWORD is required" }).min(8, "MINIO_PASSWORD must be at least 8 characters"),
SNAPSHOT_BUCKET: z.string().optional().transform((value) => value || "snapshots"),
// Tunable (SPEC §2.9): every N ops, every T seconds while peers are connected, and always on last leave.
SNAPSHOT_EVERY_OPS: positive("SNAPSHOT_EVERY_OPS", 500),
SNAPSHOT_EVERY_SECONDS: positive("SNAPSHOT_EVERY_SECONDS", 30),
```

FROM docker-compose.yml

```
minio:
  image: quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z
  command: ["server", "/data"]
  environment:
    MINIO_ROOT_USER: noon
    MINIO_ROOT_PASSWORD: "${MINIO_PASSWORD:?run ./init.sh}"
    # 9005 on the host: 9000 is a popular default (and learning-tests/minio takes 9100).
  ports: ["127.0.0.1:${MINIO_PORT:-9005}:9000"]
  volumes: ["miniodata:/data"]
```

FROM apps/sync/src/main.ts

```
const snapshots = s3Snapshots(config.minio);
await snapshots.ensureBucket(); // fails the start, like a missing variable: a sync without MinIO cannot open a snapshot document
// Not awaited to be ready: with Redis away this node still starts and answers "try again" (4503) until it returns.
const leases = createLeases({ redisUrl: config.lease.redisUrl, ttlMs: config.lease.ttlMs });
const server = await startSyncServer({ port: config.port, secrets: config.secrets, store: db.documentStore(), snapshots, cadence:
config.cadence, lease: { leases, nodeId: config.lease.nodeId }, roles: (orgId, documentId, userId) => db.roleIn(orgId, documentId, userId)
});
// E8.2 (F24): the api announces role changes on Redis; every live session they concern reads its role again.
const access = accessSubscriber({ redisUrl: config.lease.redisUrl, onChange: (change) => void server.recheck(change) });
process.stdout.write(`sync ${config.lease.nodeId} listening on ${server.url}\n`);
```

FROM apps/sync/src/snapshots.test.ts

```
test("one write at a time: triggers during a write are dropped, and take waits for it, then writes what is newer", async () => {
  const { room, written, control, snap, accept } = rig({ everyOps: 1 });
  let release = (): void => undefined;
  control.hold = new Promise((resolve) => { release = resolve; });
  accept(); // starts a write of 1, held
  accept(); accept(); // dropped: a write is running
  const last = snap.take(); // what the last peer leaving does
  await flush();
  expect(written).toEqual([]);
  control.hold = undefined;
  release();
  expect(await last).toBe(true);
  expect(written).toEqual([1, 3]);
  expect(room.seq).toBe(3);
  snap.stop();
});
```

FROM apps/sync/src/snapshot.int.test.ts

```
// integration:snapshot-replay-only-later-rows
test("opening loads the newest snapshot and replays only the journal rows after it", async () => {
  const doc = await aDocument();
  const first = [add("a"), add("b"), gap("a", 1), gap("b", 2), add("c"), gap("c", 3), gap("a", 4)];
  let server = await start(countingStore());
  const peer = await connect(server.url, doc.id, randomUUID(), {}, doc.orgId);
  await peer.next("welcome");
  await edit(peer, first);
  peer.close();
  await until(() => server.roomCount() === 0, "the last leave to snapshot and drop the room");
  await server.idle();
  await server.close();

  // Stored in MinIO under the zero-padded key, THEN named in Postgres.
  const at7 = first.reduce<Doc>(applyOp, emptyDoc());
  expect(await snapshotSeq(doc.id)).toBe(7);
  const bytes = await snapshots.get(doc.orgId, doc.id, 7);
  expect(bytes && decodeSnapshot(bytes)).toEqual(at7);
  // IfNoneMatch: a second writer of the same key changes nothing.
  await snapshots.put(doc.orgId, doc.id, 7, encodeSnapshot(emptyDoc()));
  const again = await snapshots.get(doc.orgId, doc.id, 7);
  expect(again && decodeSnapshot(again)).toEqual(at7);

  // Three more ops journaled after the snapshot, by a room that died before its next one.
  const later = [gap("b", 5), add("d"), gap("d", 6)];
  const journal = t.db.documentStore();
  for (const [i, op] of later.entries()) await journal.append(doc.orgId, doc.id, { seq: 8 + i, opId: randomUUID(), actor: { kind: "user", id: "crashed" }, op });

  const store = countingStore();
  server = await start(store);
  try {
    const back = await connect(server.url, doc.id, randomUUID(), {}, doc.orgId);
    expect(await back.next("welcome")).toMatchObject({ seq: 10, doc: later.reduce<Doc>(applyOp, structuredClone(at7)) });
    expect(store.replays).toEqual([{ after: 7, rows: 3 }]); // the 7 rows under the snapshot were never read
    back.close();
  } finally {
    await server.close();
  }
});
```

FROM apps/sync/src/snapshot.int.test.ts

```
// integration:open-10k-under-2s (F19)
test("a document with 10,000 journaled ops and no snapshot opens in under 2 s; after its snapshot, with no replay at all", async () => {
  const doc = await aDocument();
  const nodes = Array.from({ length: 100 }, (_, i) => `n${String(i)}`);
  const ops: Op[] = [...nodes.map(add), ...Array.from({ length: 9900 }, (_, i) => gap(nodes[i % nodes.length] ?? "n0", i))];
  await t.rawQueryQuery(
    `insert into op_journal (document_id, org_id, seq, op_id, actor_kind, actor_id, op)
    select $1, $2, r.seq, gen_random_uuid(), 'user', 'bulk', r.op from jsonb_to_recordset($3::jsonb) as r(seq bigint, op jsonb)`,
    [doc.id, doc.orgId, JSON.stringify(ops.map((op, i) => ({ seq: i + 1, op })))],
  );
  const expected = ops.reduce<Doc>(applyOp, emptyDoc());

  const store = countingStore();
  const server = await start(store);
  try {
    let began = performance.now();
    const cold = await connect(server.url, doc.id, randomUUID(), {}, doc.orgId);
    const welcome = await cold.next("welcome");
    const coldMs = performance.now() - began;
    expect(welcome).toMatchObject({ seq: 10_000, doc: expected });
    expect(store.replays).toEqual([{ after: 0, rows: 10_000 }]);
    expect(coldMs, `opened in ${coldMs.toFixed(0)} ms`).toBeLessThan(2000);
    cold.close();
    await until(() => server.roomCount() === 0, "the last leave to snapshot and drop the room");
    await server.idle();
    expect(await snapshotSeq(doc.id)).toBe(10_000);

    began = performance.now();
    const warm = await connect(server.url, doc.id, randomUUID(), {}, doc.orgId);
    expect(await warm.next("welcome")).toMatchObject({ seq: 10_000, doc: expected });
    const warmMs = performance.now() - began;
    expect(store.replays.at(-1)).toEqual({ after: 10_000, rows: 0 });
    expect(warmMs, `opened in ${warmMs.toFixed(0)} ms`).toBeLessThan(2000);
    warm.close();
  } finally {
    await server.close();
  }
});
```

<pre> } }); </pre>		
TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>gzipSync(JSON.stringify(doc)) → Uint8Array</code>	<code>GZIPOutputStream</code> over a <code>ByteArrayOutputStream</code>	<code>gzip.compress(json.dumps(doc).encode())</code>
<code>PutObjectCommand({ IfNoneMatch: "*" }); 412 is err.name === "PreconditionFailed"</code>	SDK v2 <code>PutObjectRequest.builder().ifNoneMatch("*"); S3Exception.statusCode() == 412</code>	<code>put_object(IfNoneMatch="*"); ClientError with code PreconditionFailed</code>
<code>String(seq).padStart(16, "0")</code>	<code>String.format("%016d", seq)</code>	<code>f"{seq:016d}"</code>
<code>while (writing) await writing;</code>	a <code>ReentrantLock</code> , or wait on a <code>CompletableFuture</code> field	<code>while self.writing: await self.writing</code>
<code>setInterval(...).unref(); timer.unref()</code>	a daemon thread on a <code>ScheduledExecutorService</code>	an <code>asyncio</code> task cancelled at shutdown
<code>res.Body.transformToByteArray() :</code> the SDK's body is a stream	<code>ResponseBytes.asByteArray()</code>	<code>obj["Body"].read()</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The document was read after an `await`, and the test that reads back the snapshot caught it.

The first draft of `take` awaited the previous write and then called `write(seq, encodeSnapshot(room.doc))` with `seq` read before the `await`. Under the unit test's held write, the room moved on and the snapshot named 1 held `seq` 3's document. The fix is one line's position, and the comment above it exists so nobody moves it back. The integration test does the same check against MinIO: the bytes at key 7 decode to the document after exactly seven ops.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A snapshot store inside `packages/db` was rejected.

It would have been convenient: one store with `load`, `append`, `snapshot`. But `packages/db` is Postgres, and SPEC keystone 9 gives each store one job. S3 lives in `apps/sync`, behind a four-line `SnapshotStore` type that the tests implement in memory. The pointer, which *is* relational, stays in `packages/db`.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The listing that would have found the newest snapshot pages forward, 1000 keys at a time, for ever.

Measured in the learning test with 1005 objects: the newest key is on page two, and `MaxKeys: 5000` does not change that. A design that listed to find the latest snapshot would get slower with every snapshot ever taken. The pointer row costs one `update` per snapshot and one column per document, and turns "find the newest" into "build the key".

Principal-level point: a snapshot is a materialised prefix of the log. Everything about it follows from that: it may be lost (rebuild from the log), it must never be *wrong* (read the document in the same tick as its `seq`), two writers may race for it (conditional write, and equal content makes the loser's loss harmless), and the pointer to it is the only thing that must be consistent (write it after, move it forward only). Kafka Streams checkpoints, Raft snapshots and database checkpoints are the same four rules with other names.

5 · Proving the promise: a ledger, the journal, and two faults on the real stack

SPEC §4a insists that durability is verified against real processes, not simulated. Epic 6 writes two chaos checks and, more important, writes the *assertion* once as a pure module so that epic 7's failover run can use it unchanged. The idea: while the run makes edits, keep a **ledger** of every op each peer submitted and what it knew about that op at the moment of the fault. Afterwards, read the journal (the truth) and hold the ledger against it.

FROM `scripts/chaos/no-loss.ts`

```

// F18's guarantees as one pure check, reused by every chaos run that breaks the sync server mid-edit:
// chaos:kill-sync-no-loss (E6.3) now, the failover run (E7.2) next. The run keeps a LEDGER of every op

```

```
// its peers submitted and when each settled; afterwards it reads the journal (the truth) and asks:
// - every op acknowledged before the fault is journaled, at the seq it was acknowledged with;
// - every op in flight at the fault (or made while the server was gone) was resent and applied once;
// - no op is journaled twice, the journal's seqs are 1..n without a gap, nothing untracked is in it;
// - every peer ends on the same confirmed document.
// Vacuity guards (SPEC §4a): a run with no op acknowledged before the fault, or none in flight at it,
// proved nothing, and says so.
```

FROM `scripts/chaos/no-loss.ts`

```
export function createLedger() {
  const entries: LedgerEntry[] = [];
  const settling: Promise<unknown>[] = [];
  let faulted = false;
  return {
    entries: entries as readonly LedgerEntry[],
    track(peer: string, submitted: Submitted): void {
      if (!submitted.ok) return;
      const entry: LedgerEntry = { peer, opId: submitted.opId, atFault: faulted ? "after" : "unsettled", outcome: "unsettled" };
      entries.push(entry);
      settling.push(submitted.settled.then((outcome) => { entry.outcome = outcome; }));
    },
    /** Call right after the fault is opened: what has settled by now counts as "before the fault". */
    fault(): void {
      faulted = true;
      for (const entry of entries) if (entry.atFault === "unsettled") entry.atFault = entry.outcome;
    },
    /** Wait for every op to settle, at most `timeoutMs`; whatever has not by then stays "unsettled" (lost). */
    async settle(timeoutMs: number): Promise<void> {
      let timer: NodeJS.Timeout | undefined;
      await Promise.race([Promise.all(settling), new Promise((resolve) => { timer = setTimeout(resolve, timeoutMs); })]);
      clearTimeout(timer);
    },
  };
}
```

FROM `scripts/chaos/no-loss.ts`

```
/** Every broken guarantee, in words; empty = PASS. `docs`: each peer's confirmed document, serialized. */
export function noLossViolations({ ledger, journal, docs }: { ledger: readonly LedgerEntry[]; journal: readonly JournalRow[]; docs:
readonly string[] }): string[] {
  const violations: string[] = [];
  if (!ledger.some((e) => typeof e.atFault === "object" && e.atFault.ok && e.atFault.seq !== undefined)) violations.push("vacuous: no op
was acknowledged before the fault");
  if (!ledger.some((e) => e.atFault === "unsettled")) violations.push("vacuous: no op was in flight at the fault, so nothing was resent");

  const seqs = journal.map((row) => row.seq);
  if (seqs.some((seq, i) => seq !== i + 1)) violations.push(`the journal's seqs are ${seqs.join(",")}, not 1..${String(seqs.length)}
without a gap or repeat`);
  const seqsOf = new Map<string, number[]>();
  for (const row of journal) seqsOf.set(row.opId, [...(seqsOf.get(row.opId) ?? []), row.seq]);
  for (const [opId, at] of seqsOf) if (at.length > 1) violations.push(`op ${opId} is journaled ${String(at.length)} times (seq
${at.join(", ")} )`);
  const tracked = new Set(ledger.map((e) => e.opId));
  for (const row of journal) if (!tracked.has(row.opId)) violations.push(`journal seq ${String(row.seq)} holds op ${row.opId}, which no
tracked peer submitted`);

  const bySeq = new Map(journal.map((row) => [row.seq, row.opId]));
  for (const e of ledger) {
    const refusedBefore = typeof e.atFault === "object" && !e.atFault.ok;
    const what = `${e.peer} op ${e.opId}${refusedBefore ? "" : e.atFault === "unsettled" ? " (in flight at the fault)" : e.atFault ===
"after" ? " (made after the fault)" : " (acknowledged before the fault)"}`;
    if (e.outcome === "unsettled") violations.push(`${what} never settled: lost`);
    else if (!e.outcome.ok) {
      if (!refusedBefore) violations.push(`${what} was refused (${e.outcome.reason}): lost`);
      else if (seqsOf.has(e.opId)) violations.push(`${what} was refused (${e.outcome.reason}) but is journaled`);
    } else if (e.outcome.seq !== undefined && bySeq.get(e.outcome.seq) !== e.opId) {
      violations.push(`${what} was acknowledged at seq ${String(e.outcome.seq)}, but the journal has ${bySeq.get(e.outcome.seq) ??
"nothing"} there`);
    }
  }

  docs.forEach((doc, i) => { if (doc !== docs[0]) violations.push(`peer ${String(i)}'s confirmed document differs from peer 0's`); });
  return violations;
}
```

This is what "no loss" means *precisely*, and it is worth saying in words, because an interviewer will ask. An op **acknowledged before the fault** must be in the journal at exactly the seq it was acknowledged with. An op **in flight at the fault** (sent, unanswered) or **made while the server was gone** must settle `ok` and appear exactly once. Nothing may be refused for a reason the run did not cause, and nothing refused before the fault may be in the

journal. The journal's seqs are `1..n` with no gap or repeat, no opId appears twice, and every row belongs to a tracked peer. And every peer's confirmed document is byte-identical. Then two **vacuity guards**: a run in which nothing was acknowledged before the fault, or nothing was in flight at it, proved nothing, and says so as a violation. SPEC §4a's rule, "every `always` has a `sometimes`", is that pair of lines.

The `Outcome` and `Submitted` types are copied from peer-client's public shape rather than imported, so the module depends on nothing. `ledger.fault()` is the one moment that matters: whatever has *settled* by then counts as "before", everything else is "in flight". `settle(timeoutMs)` is `Promise.race` between "every op settled" and a timer, with the timer cleared afterwards, so an op that never settles is left as `"unsettled"` and reported as lost rather than hanging the run.

FROM `scripts/chaos/kill-sync-no-loss.ts`

```
/** One document, one kill. Returns what broke, in words (empty = PASS), and how much was exercised. */
async function round(n: number): Promise<{ round: number; ops: number; journaled: number; violations: string[] }> {
  const doc = idOf(await post(`/orgs/${org}/workspaces/${workspace}/documents`, { title: `chaos kill sync ${String(n)}` }));
  const session = async () => SessionResponse.parse(await post(`/documents/${doc}/session`));
  const peers = {
    person: connectPeer({ manifest, session }),
    // The api's address (the room's owner, E7.1) with the worker's kind of token: the AI routes as the worker does.
    ai: connectPeer({ manifest, session: async () => ({ wsUrl: (await session()).wsUrl, token: signSessionToken({ userId: me, orgId: org,
documentId: doc, secret, ttlSeconds: 600, actor: { kind: "agent", runId: randomUUID() } }) })),
  };
  const all = Object.entries(peers);
  const ledger = createLedger();
  let made = 0;
  let victim = "sync";
  const edit = (count: number): void => {
    for (let i = 0; i < count; i++) for (const [name, peer] of all) ledger.track(name, peer.submit(add(`${name}-${String(made++)}`)));
  };
  try {
    await until(() => all.every([, p]) => p.status === "live"), "both peers live", 15_000);
    edit(5);
    await ledger.settle(10_000); // acknowledged before the fault

    victim = ownerOf(doc);
    edit(20); // on the wire...
    compose("kill", "--signal", "SIGKILL", victim); // ...and the server dies under it. execFileSync: no ack is heard meanwhile.
    ledger.fault();
    await until(() => all.every([, p]) => p.status !== "live"), "both peers see the server gone", 30_000);
    edit(5); // made while the server is gone: held, sent after the next welcome

    // Back at once: the peers land on it or on the other node (E7.2); either waits the dead lease out (LEASE_TTL_MS).
    compose("start", victim);
    await until(() => all.every([, p]) => p.status === "live" && p.pendingCount === 0), "both peers reconnected unaided, nothing
pending", 60_000);
    await ledger.settle(15_000);
    await until(() => all.every([, p]) => p.seq === peers.person.seq), "both peers at the same seq", 10_000);

    const journal = psql(`select seq || ' ' || op_id from op_journal where document_id = '${doc}' order by seq`)
      .split("\n").filter(Boolean).map((line) => { const [seq, opId] = line.split(" "); return { seq: Number(seq), opId: opId ?? "" }; });
    const docs = all.map([, p] => JSON.stringify(p.confirmed));
    return { round: n, ops: ledger.entries.length, journaled: journal.length, violations: noLossViolations({ ledger: ledger.entries,
journal, docs }) };
  } finally {
    for (const [, p] of all) p.close();
    compose("start", victim); // whatever failed above, the server comes back
  }
}
```

The kill is a *synchronous* `docker compose kill --signal SIGKILL sync` through `execFileSync`, called right after a burst of twenty edits per peer. Because the call blocks the event loop, no acknowledgement can be processed between the last `submit` and the kill: the burst is guaranteed in flight, and some of it is in the state the journal's dedupe exists for (journaled, acknowledgement never heard). The server is started again and the *peers are left alone*: F18 says they reconnect unaided, so the run waits on `status === "live" && pendingCount === 0` and nothing else. Each round uses a fresh document and one fault (several kills on one ledger would blur "acknowledged before *which* fault"), and `ROUNDS=3` repeats, because a one-shot pass proves nothing. `finally` starts sync again whatever happened.

FROM `scripts/chaos/kill-sync-no-loss.ts`

```
const secretOrNone = process.env["SESSION_TOKEN_SECRET"] ?? (existsSync(".env") ? parseEnv(readFileSync(".env", "utf8"))
["SESSION_TOKEN_SECRET"] : undefined);
if (secretOrNone === undefined) throw new Error("SESSION_TOKEN_SECRET is not set and .env has none: run ./init.sh");
const secret: string = secretOrNone; // round() is a function declaration: the narrowing above does not reach into it
const env = { ...process.env, PATH: `${process.env["PATH"]} ?? "":/Applications/Docker.app/Contents/Resources/bin` };
const compose = (...args: string[]): string => execFileSync("docker", ["compose", ...args], { env, encoding: "utf8" });
const psql = (sql: string): string => compose("exec", "-T", "postgres", "psql", "-U", "noon", "-d", "noon", "-tAc", sql).trim();
```

A TypeScript detail worth the two lines: narrowing does not cross a *function declaration*. `secretOrNone` is `string | undefined`; the `if (... === undefined)` `throw` narrows it to `string` in the top-level code, but `round()` is a hoisted `function` that could, as far as the compiler knows, run before the check. So a second `const secret: string` carries the narrowed value in. Java has no such issue (a final field is assigned once); Python has no such check.

FROM `scripts/chaos/postgres-down-read-only.ts`

```
/** One outage: `down` takes Postgres away, `up` brings it back; `name` names this round and its nodes. */
async function outage(name: string, down: () => void, up: () => void, readOnlyWithinMs: number): Promise<void> {
  const seqBefore = peers.person.seq;
  down(); // the fault first, then the workload (a fast pipeline outruns a late fault)
  try {
    const held = peers.person.submit(add(`held-${name}`));
    check(held.ok, `${name}: the person's edit is accepted locally`);
    await until(() => all.every((p) => p.readOnly), `${name}: every peer shows read-only`, readOnlyWithinMs);
    for (const [who, peer] of [["AI", peers.ai], ["git peer", peers.git]] as const) {
      const tried = peer.submit(add(`${who.replace(" ", "-")}-${name}`));
      check(!tried.ok && tried.reason === "read_only", `${name}: the ${who}'s edit is refused as read_only`);
    }
    check(all.every((p) => p.status === "live"), `${name}: every peer stays connected`);
    check(peers.person.pendingCount === 1 && peers.person.seq === seqBefore, `${name}: the held edit is pending, nothing acknowledged`);
    steps.push(`${name}: read-only on all three peers`);
    up();
    await until(() => all.every((p) => !p.readOnly), `${name}: every peer writable again`, 60_000);
    const outcome = held.ok ? await held.settled : undefined;
    check(outcome?.ok === true && outcome.seq === seqBefore + 1, `${name}: the held edit landed once, at seq ${String(seqBefore + 1)} (got ${JSON.stringify(outcome)})`);
    await until(() => all.every((p) => p.seq === seqBefore + 1), `${name}: every peer at seq ${String(seqBefore + 1)}`, 10_000);
    steps.push(`${name}: recovered, held edit at seq ${String(seqBefore + 1)}`);
  } finally {
    up(); // whatever failed above, Postgres comes back
  }
}
```

The second run is section 3 against the real stack, twice: Postgres *stopped* (connections refused, read-only within a round trip) and *paused* (queries hang, read-only after the 5 s journal timeout). Note the §4a pitfall the code names: the fault is opened *before* the workload, because a fast pipeline outruns a late fault. Every wait polls an observable (`peer.readOnly`, `peer.seq`) with a deadline, never a fixed sleep. And the whole thing prints one JSON line and exits non-zero on FAIL, so `make chaos` can run it in a loop.

FROM `Makefile`

```
# Outside `make check`: the chaos checks (SPEC §4a) break the REAL compose stack (./init.sh first) and prove
# the system's guarantees hold. Each prints one JSON line and exits non-zero on FAIL.
.PHONY: chaos
chaos: ; node scripts/chaos/postgres-down-read-only.ts && node scripts/chaos/kill-sync-no-loss.ts && node scripts/chaos/kill-owner-failover.ts && node scripts/chaos/fenced.ts zombie && node scripts/chaos/fenced.ts partition && node scripts/chaos/kill-worker-resumes.ts && node scripts/chaos/redis-wipe-rebuild.ts
```

FROM `scripts/chaos/no-loss.test.ts`

```
it("refuses a run that proved nothing (vacuity guards)", () => {
  expect(noLossViolations({ ledger: [acked], journal: [first], docs: same })).toEqual(["vacuous: no op was in flight at the fault, so nothing was resent"]);
  expect(noLossViolations({ ledger: [{ ..inFlight, outcome: { ok: true, seq: 1 } }], journal: [{ seq: 1, opId: "b" }], docs: same })).toEqual(["vacuous: no op was acknowledged before the fault"]);
});
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>execFileSync("docker", ["compose", "kill", ...]):</code> synchronous, argument array, no shell	<code>ProcessBuilder(...).start().waitFor()</code>	<code>subprocess.run(...,</code> <code>check=True)</code>
<code>const secret: string = secretOrNone</code> to carry a narrowing into a hoisted function	not needed: fields have one type	not needed: no checker
<code>Awaited<ReturnType<typeof round>></code> : the type of what an async function resolves to	the <code>T</code> of a <code>CompletableFuture<T></code> , spelled out	the return annotation, spelled out
top-level <code>await</code> in a script run by <code>node file.ts</code>	a <code>main</code> method	<code>asyncio.run(main())</code>
<code>process.exit(verdict === "PASS" ? 0 : 1)</code> after one JSON line on stdout	<code>System.exit</code>	<code>sys.exit</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Several kills on one ledger were rejected.

The tempting run is one document, kill, kill, kill. But the ledger's central question is "what was acknowledged *before the fault*", and with three faults an op acknowledged after the first and lost at the second has no single answer. One fault per document per round keeps every entry's `atFault` meaningful, and `ROUNDS` gives the repetition \$4a asks for.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The assertion is unit-tested on synthetic ledgers, because the chaos run cannot run here.

`no-loss.test.ts` feeds `noLossViolations` hand-written ledgers and journals: a clean run, a lost ack, an op journaled twice, a gap, an unsettled op, a refused op that is in the journal anyway, peers that diverge, and both vacuity guards. It is the cheapest way to know the checker itself is not vacuous before it is trusted against the stack. The unit layer's `include` gained `scripts/**/*.test.ts` for it, and knip treats `scripts/chaos/*.ts` as entries so their exports are not reported dead.

The Antithesis habit, in one sentence: write the property as a pure function over a ledger and the truth; make the run's only job be to break the system while the ledger fills; refuse a run that did not exercise the property. Everything in `scripts/chaos/` is that sentence, and epic Z's harness will call the same function.

6 · Testing durability without a database, and then with one

Every rule in this chapter has a test that needs no Docker, and a second one that needs the compose stack. The first kind is what ran on every commit; the second kind is written and unrun (the status block at the top says so). Three shapes recur.

A stand-in for the table, with the table's two unique keys. `memoryJournal` is thirty lines that behave as `op_journal` does: a resend returns the original row, a taken seq rejects, `since` filters and sorts. Because it outlives any room built on it, a test can build a second room on the same journal and stand for a restart.

FROM `apps/sync/src/room.test.ts`

```
/** A journal in memory with the table's two unique keys, outliving any room built on it: a stand-in for op_journal. */
function memoryJournal(): Journal & { rows: SequencedOp[] } {
  const rows: SequencedOp[] = [];
  const find = (actorId: string, opId: string) => rows.find((r) => r.actor.id === actorId && r.opId === opId);
  return {
    rows,
    append(op) {
      const original = find(op.actor.id, op.opId);
      if (original) return Promise.resolve(original);
      if (rows.some((r) => r.seq === op.seq)) return Promise.reject(new Error("op_journal_seq"));
      rows.push(op);
      return Promise.resolve(undefined);
    },
    find: (actorId, opId) => Promise.resolve(find(actorId, opId)),
    everAdded: (nodeId) => Promise.resolve(rows.some((r) => r.op.type === "add_node" && r.op.nodeId === nodeId)),
    since: (seq) => Promise.resolve(rows.filter((r) => r.seq > seq).sort((x, y) => x.seq - y.seq)),
  };
}
```

A stand-in behind a switch. `flakyJournal` wraps the memory journal in a `guard` that rejects while `down`, plus one deliberately nasty mode, `committedAnyway`: the next append lands *and* throws, which is the lost-reply case section 3's recovery exists for. The integration file's `switchableStore` adds `hang`, a promise that never settles, for the paused-database case. The generic `guard` is a higher-order function typed with a tuple rest parameter (`<A extends unknown[], R>(call: (...args: A) => Promise<R>) => (...args: A): Promise<R> => (state.down ? Promise.reject(new Error("db down")) : call(...args))`): it wraps any async function and keeps its signature.

FROM `apps/sync/src/room.test.ts`

```
// --- E6.1b: with its storage unavailable the room is read-only, says so to every peer, and recovers ---
/** memoryJournal behind a switch: while 'down', every call rejects; 'committedAnyway' makes the next append land AND reject (a lost reply). */
function flakyJournal() {
  const inner = memoryJournal();
  const state = { down: false, committedAnyway: false, appends: 0 };
  const guard = <A extends unknown[], R>(call: (...args: A) => Promise<R>) => (...args: A): Promise<R> => (state.down ? Promise.reject(new Error("db down")) : call(...args));
  const journal: Journal = {
```

```

append: async (op) => {
  state.append++;
  if (state.committedAnyway) {
    state.committedAnyway = false;
    await inner.append(op);
    throw new Error("connection lost after commit");
  }
  return guard((each: SequencedOp) => inner.append(each))(op);
},
find: guard((actorId: string, opId: string) => inner.find(actorId, opId)),
everAdded: guard((nodeId: string) => inner.everAdded(nodeId)),
since: guard((seq: number) => inner.since(seq)),
};
return { journal, state, rows: inner.rows };
}
const statuses = (p: { inbox: ServerMessage[] }) => p.inbox.filter((m) => m.type === "status");

```

FROM `apps/sync/src/read-only.int.test.ts`

```

/** op_journal in memory, with its two unique keys. `down`: every call refuses. `hang`: every call waits for ever. */
function switchableStore() {
  const rows: SequencedOp[] = [];
  const state = { down: false, hang: false };
  const gate = <T>(work: () => T): Promise<T> =>
    state.hang ? new Promise<T>(() => undefined) : state.down ? Promise.reject(new Error("connection refused")) : Promise.resolve(work());
  const store: DocumentStore = {
    load: () => gate(() => ({ doc: undefined, seq: 0, snapshotSeq: 0 })),
    snapshotted: () => gate(() => undefined),
    fence: () => gate(() => 0),
    claim: () => gate(() => true),
    append: (_org, _doc, op) => gate(() => {
      const original = rows.find((r) => r.actor.id === op.actor.id && r.opId === op.opId);
      if (original) return original;
      if (rows.some((r) => r.seq === op.seq)) throw new Error("op_journal_seq");
      rows.push(op);
      return undefined;
    }),
    find: (_org, _doc, actorId, opId) => gate(() => rows.find((r) => r.actor.id === actorId && r.opId === opId)),
    everAdded: (_org, _doc, nodeId) => gate(() => rows.some((r) => r.op.type === "add_node" && r.op.nodeId === nodeId)),
    since: (_org, _doc, seq) => gate(() => rows.filter((r) => r.seq > seq).sort((a, b) => a.seq - b.seq)),
  };
  return { store, rows, state };
}

```

The real store, observed. `countingStore` spreads the real `documentStore()` into a new object and replaces one method, `since`, with a version that records what it was asked and how much it returned. That is how the integration test proves "only the rows after the snapshot were read": `replays equals [{ after: 7, rows: 3 }]`, and after a snapshot at 10,000, `[{ after: 10_000, rows: 0 }]`. Object spread over a closure-based store is the TypeScript way to decorate one method; Java would subclass or proxy.

FROM `apps/sync/src/snapshot.int.test.ts`

```

/** The real store, with every `since` it answers written down: which seq it was asked from, how many rows it gave. */
function countingStore(): DocumentStore & { replays: { after: number; rows: number }[] } {
  const real = t.db.documentStore();
  const replays: { after: number; rows: number }[] = [];
  return {
    ...real,
    replays,
    since: async (orgId, documentId, seq) => {
      const rows = await real.since(orgId, documentId, seq);
      replays.push({ after: seq, rows: rows.length });
      return rows;
    },
  };
}

```

A clock you wind by hand. The cadence tests use `vi.useFakeTimers()` and `advanceTimersByTimeAsync`, so "every 30 seconds" runs in milliseconds and the assertion "never with nobody here" is exact. The `rig` builds a room stand-in whose `seq` a test moves by hand, and a `write` that records, fails, or waits on a promise the test releases.

FROM `apps/sync/src/snapshots.test.ts`

```

/** A room stand-in whose seq a test moves by hand, and a write that records or fails on demand. */
function rig({ from = 0, everyOps = 3, everyMs = 1000 } = {}) {
  const room = { seq: from, doc: emptyDoc(), peerCount: 1 };
  const written: number[] = [];
  const control = { fail: false, hold: undefined as Promise<void> | undefined };
  const snap = snapshotter({

```

```

    room,
    from,
    cadence: { everyOps, everyMs },
    write: async (seq, body) => {
      await control.hold;
      if (control.fail) throw new Error("minio down");
      expect(decodeSnapshot(body)).toBeDefined();
      written.push(seq);
    },
  });
  const accept = (): void => { room.seq++; snap.accepted(room.seq); };
  return { room, written, control, snap, accept };
}
const flush = () => new Promise((resolve) => setImmediate(resolve));

```

FROM `apps/sync/src/snapshots.test.ts`

```

test("every T while peers are connected, only if the room moved; never with nobody here", async () => {
  vi.useFakeTimers();
  const { room, written, snap } = rig({ everyOps: 1000, everyMs: 1000 });
  await vi.advanceTimersByTimeAsync(1000);
  expect(written).toEqual([]); // nothing new: no write
  room.seq = 2;
  await vi.advanceTimersByTimeAsync(1000);
  expect(written).toEqual([2]);
  room.seq = 5;
  room.peerCount = 0;
  await vi.advanceTimersByTimeAsync(3000);
  expect(written).toEqual([2]); // the last-leave snapshot is the server's to take, not the timer's
  room.peerCount = 1;
  await vi.advanceTimersByTimeAsync(1000);
  expect(written).toEqual([2, 5]);
  snap.stop();
  room.seq = 9;
  await vi.advanceTimersByTimeAsync(5000);
  expect(written).toEqual([2, 5]);
});

```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`{ ...real, since: async (...) => { ... } }`: decorate one method by spread

a subclass or a `Proxy`; Mockito's `spy`

`unittest.mock.patch.object`

`<A extends unknown[], R>(call: (...args: A) => Promise<R>)`

no variadic generics; one wrapper per arity

`ParamSpec` in typing

`vi.useFakeTimers(); await vi.advanceTimersByTimeAsync(1000)`

an injected `Clock`; Awaitility

`freezegun`; an injected clock

a promise the test holds and releases (`control.hold`)

a `CountDownLatch`

an `asyncio.Event`

`new Promise<T>(() => undefined)`: a promise that never settles, for "hangs"

a `CompletableFuture` never completed

`asyncio.Future()` never set

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The gate's unit layer did not include `scripts/`, so the chaos checker's tests would not have run.

`vitest.config.ts`'s `include` covered `apps` and `packages`. The checker lives in `scripts/chaos/`, next to the run that uses it, so the pattern gained `scripts/**/*.test.ts`. A test that is written and not collected is the most misleading kind of green there is; check the collector when you add a test somewhere new.

Honest status, repeated here because it matters: the unit tests in this chapter (the room's journal, read-only and recover rules; peer-client's hold; the snapshotter's cadence and key rules; the chaos checker) ran green on every commit. The integration suites (`journal.int`, `read-only.int`, `snapshot.int`, `packages/db`'s `journal.int`, and the E6.1b additions to `push.test` and `git.int`) and both chaos runs are written, reviewed and committed, and have not been run. The next section is what running them looks like.

7 · Durability by hand: what to run once the stack is up

These are the exact commands and what each should show, in the order that builds confidence. The dev stack gains MinIO (`127.0.0.1:9005`, no console published) and the sync container refuses to start without it. Nothing here needs a token; where a document id is needed, it is the `?doc=` in the canvas's address bar.

0 · BRING THE STACK UP

```
./init.sh # now also MinIO; sync makes the bucket "snapshots" at start and refuses to start without it
docker compose ps # minio healthy; sync healthy
open http://localhost:5173/?user=you@example.com # New document; Add Stack, Add Button, drag a little; note ?doc=<id>
```

1 · THE JOURNAL IS THE TRUTH

```
docker compose exec -T postgres psql -U noon -d noon -c \
  "select seq, actor_kind, op->>'type' as op, left(op_id::text, 8) as op_id from op_journal where document_id = '<id>' order by seq"
# one row per accepted op, seq 1..n with no gap; the drag is many set_prop rows
```

2 · KILL -9 MID-EDIT (F18)

```
# In the canvas, start dragging a node and keep dragging while you run:
docker compose kill --signal SIGKILL sync # the status turns "offline" with "saving N..."; keep editing: edits queue locally
docker compose start sync # within seconds, without a reload: "live", then "saved"; the other tab shows the same page
# Then the query from step 1 again: n grew by exactly the edits you made; no seq repeats; no op_id repeats:
docker compose exec -T postgres psql -U noon -d noon -c \
  "select count(*), count(distinct seq), count(distinct op_id), max(seq) from op_journal where document_id = '<id>'" # all four equal
```

3 · POSTGRES DOWN: READ-ONLY, NOTHING LOST (E6.1B)

```
docker compose stop postgres # add a node NOW: the status says "read-only · waiting to save 1...", the alert appears, Add is
disabled # a second tab opened meanwhile is welcomed read-only
docker compose start postgres # within about a second: "live · saved"; the node you added is there, once, in both tabs
docker compose pause postgres # the same, but read-only arrives after the 5 s journal timeout (queries hang instead of
failing)
docker compose unpause postgres
```

4 · SNAPSHOTS (F19)

```
docker compose exec -T postgres psql -U noon -d noon -c "select snapshot_seq, seq from documents where id = '<id>'"
# snapshot_seq moves every 500 ops, every 30 s while a tab is open, and when the last tab
closes # the object is <org>/<id>/<16-digit seq>.json.gz in the bucket; no console is published, the
integration test reads it back
SNAPSHOT_EVERY_OPS=5 docker compose up -d sync # to watch the cadence by hand: every five edits moves the pointer
docker compose logs sync | grep snapshot # only FAILED snapshots are logged; silence is the good case
```

5 · THEN THE CHECKS THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run --config vitest.integration.config.ts packages/db/src/journal.int.test.ts apps/sync/src/journal.int.test.ts
apps/sync/src/read-only.int.test.ts apps/sync/src/snapshot.int.test.ts
# real Postgres and MinIO; the 10,000-row open is timed and must be under 2 s
node scripts/chaos/postgres-down-read-only.ts # one JSON line: {"chaos":"postgres-down-read-only","verdict":"PASS","steps":[...]}
ROUNDS=3 node scripts/chaos/kill-sync-no-loss.ts # {"chaos":"kill-sync-no-loss","verdict":"PASS","rounds":
[{"ops":60,"journalled":...,"violations":[]},...]}
make chaos # both, in order; breaks and restores the real stack
make check # the whole gate, as the pre-commit hook runs it
```

What to do when a step does not match this page: the page was cut from the code at commit `f21d6d2`, and the code is right until a test says otherwise. A step that fails is a finding: write it as *sequence* → *false belief* → *consequence* → *smallest fix* (SPEC §4a's form) and file it against the bead that owns the code.

8 · What the type system could not see, epic 6 edition

Everything below compiled under `strict`, passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
an op announced before it was durable	the compiler cannot see the order of two awaits	append, then <code>accept</code> ; drill 2 plants the reverse
a failed write took a sequence number	<code>seq + 1</code> looked like <code>seq++</code>	the number is taken inside <code>accept</code> only
two unique violations look the same (23505)	the code is not the constraint	read <code>err.constraint</code> by name
a bare-opId key would refuse a cross-peer collision	every broadcast shows every opId	key on <code>(document, actor, opId)</code>
a forged <code>baseSeq</code> walked past the stale guard (E2.3)	memory is bounded	the journal's key answers with the original
a reopened room numbered from a stale seq	the saved document is behind the journal	replay <code>since(seq)</code> at open
a removed id came back through any actor	<code>validate</code> sees only the present	<code>everAdded</code> through a partial index
a paused database hung the room for ever	a hang is not an error	<code>bounded</code> : a timeout that means "treat as failed"
a health probe went writable with the room behind the journal	"answers" is not "agrees"	<code>recover()</code> replays, then says writable
a refusal under read-only was a resync, and a reconnect storm	<code>unavailable</code> meant one thing in lesson 2	the status message turns it into a hold
a held op that landed would be applied twice, or heard by nobody	a lost reply is not a failed write	replay before writable; the resend gets the original
a git event failed during an outage was gone for good	<code>(ref, after_sha)</code> is never re-recorded	<code>WaitAgain</code> : back to <code>pending</code>
a snapshot named <code>seq</code> held a later document	the room edits <code>doc</code> in place	encode in the same tick as <code>seq</code>
the pointer could name a missing object	two writes, one crash between them	MinIO first, then <code>snapshot_seq</code>
a slow old snapshot could move the pointer back	writes finish out of order	where <code>snapshot_seq < \$3</code>
"10" sorted before "9"	S3 keys are strings	zero-pad to 16 digits
finding the newest snapshot by listing	1000 keys a page, ascending only (measured)	a pointer row; build the key
MinIO down read as "no snapshot"	one <code>undefined</code> for two facts	<code>NoSuchKey</code> → <code>undefined</code> ; anything else throws; 4503 vs 4500
a chaos run that proved nothing passed	a green run with no op in flight	vacuity guards as violations
a kill that landed after the burst was acknowledged	an async kill lets acks through	<code>execFileSync</code> : the loop is blocked until the kill lands
a narrowing that did not reach a hoisted function	declarations hoist; narrowings do not	<code>const secret: string = secretOrNone</code>
the checker's tests were not collected	the unit pattern stopped at <code>packages</code>	<code>scripts/**/*.test.ts</code> in the include

The pattern, extended once more: lessons 2 to 5 said types prove shapes and say nothing about order, time, repetition, memory, who is speaking, or which side of a boundary a thing is on. Epic 6 adds the two that durability turns on: types cannot see *whether a write happened* (a rejected promise may have landed; a hung one may too) and *what order two effects took* (announce, then persist, compiles as well as the reverse). Both were answered the same way as before: put the truth in one place, read it before you believe anything, and write a test that watches the order.

Your turn: three drills

Do these alone. They live in `drills/lesson-6/` and both coded drills start RED; `sh drills/lesson-6/check.sh` shows your progress. Neither needs Docker.

D1 • Extend a type, end to end. The canvas says "read-only" and holds the person's edits, but not since when. Carry the moment from the room's *lent* clock to the screen: the `status` message gains a required `at` and the welcome an optional `readOnlySince` (`packages/contracts`); `storageFailed` and `recover` stamp them from `now()`, never `Date.now`, and a peer joining meanwhile is told

since when; peer-client exposes `readOnlySince` and clears it as it clears `readOnly`; `readOnlyFor(since, now)` in `apps/web/src/reasons.ts` says "5 s" or "2 min", never negative; the canvas's alert reads "Read-only for 2 min", as text. make check must stay green: the type checker will name every place a status is built without `at`, and `room.test.ts`, `peer.test.ts` and `wire.test.ts` will name every expectation that changed. That list is the drill.

Predict first: why must the room take the moment from the clock it was lent, and what would a test that winds the clock by hand be unable to check if it used `Date.now`?

D2 · Fix a planted bug. `drills/lesson-6/d2-durable-room/room.ts` is the room cut down to durability (queue, dedupe, journal, read-only, recover) with one bug from this chapter planted in it. Its first test fails the database's write and asks who heard of the op. Read the failing assertion and say, before you open the code, what the room believes its own `seq` is now and why that is worse than the lost broadcast. The fix moves one line.

D3 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. The room could apply and broadcast an op, then journal it, and refuse it if the write fails. That is faster for the sender. Why is the acknowledgement given only after the append, and why is the sequence number taken only then?

Because an acknowledgement is a promise about the future: "after a crash this will still be true". Only the database can make that promise, so the room may only relay it after the database has spoken. Announce first and three things break at once. Every other peer applies an op that may not exist after a restart, so two tabs diverge from the truth. The sender's client marks the op confirmed and forgets it, so nothing will ever resend it: the op is lost *while everyone saw it*. And the room's `seq` has moved for a row that is not in the journal, so the next append's number is one ahead of the table, which is the room's own signal for a rival writer; under the reversed order it becomes a permanent read-only after the first failed write. Taking the number inside `accept`, after the append resolved, means a failed write leaves no gap and the next op simply gets the same number. The cost is one database round trip per op inside the room's queue, which is the price of the promise. Drill 2 plants the reverse order; its first assertion is peer B having received seq 2.

▼ 2. When the journal fails, the room could ping Postgres every second and say "writable" when it answers. Instead `recover()` reads `since(seq)` and replays it first. Why, and what exactly does "no acknowledged op is lost, none is applied twice" require of the client and the journal together?

Because "the database answers" and "the room agrees with the database" are different facts. A rejected append may have landed (the connection died after the commit) and a timed-out one may land later; a rival row may sit at the room's next seq. If the room goes writable without reading, the landed op is in the journal and in nobody's document, and the next append from anyone breaks `op_journal_seq` and puts the room back into read-only for ever. Replaying through `accept` applies and announces what landed, moves the room's seq past it, and only then says writable; the sender's held resend is then answered from memory with the seq it got. The promise needs three things. From the journal: a unique key on `(document, sender, opId)`, and the original row returned when it is broken, so a second sending gets the first answer. From the client: keep the op under the same `opId` until it is answered, and re-offer it after every welcome and every "writable", in order. From the room: acknowledge only what is durable, refuse everything while it cannot write, and read the journal before it writes again. Remove any one and the promise is gone: no key, and a resend is applied twice; no held resend, and an in-flight op is lost; no replay, and a landed op is heard by nobody.

▼ 3. Snapshots are written to MinIO with `IfNoneMatch: "*"`, and `documents.snapshot_seq` is updated only afterwards and only forwards. A failed snapshot is logged and never retried, and F8's idle save was deleted. Defend each of those four choices in one sentence, then say what a snapshot is.

A snapshot is a materialised prefix of the log, so everything follows from the journal being the truth. *Conditional put*: two rooms (or one room twice, after a crash) may reach the same seq, and the same journal prefix gives the same document, so the loser of the race loses nothing and a plain overwrite would only open a window where the object is half-written. *Pointer after the object, forwards only*: a crash between the two writes leaves an orphaned object (harmless) rather than a pointer to nothing (a document that cannot open), and a slow write of an older snapshot finishing late must not move the document backwards. *Never retried*: a snapshot that failed to write costs nothing but more rows to replay at the next open, so a retry loop would be complexity spent protecting a copy. *F8 deleted*: F8 had to retry for ever because the room's memory was the only copy of the edits since the last save; with the journal that is never true, and keeping two save paths means two truths. The key is zero-padded because S3 sorts strings, and the newest snapshot is found by the pointer rather than by listing because S3 lists ascending in pages of 1000, measured. What it is: a cache of the log that may be lost but must never be wrong, which is why the document is encoded in the same tick as its seq.

Recall first

Look away. Name the three things the room does to an op after the journal accepts it, in order, and who the sender's acknowledgement is. Then name the two starting points a document can open from and what is replayed after either. Then check.

▼ Check yourself

After the append resolves, `accept` : applies the op to the in-memory document, takes the sequence number (only now), remembers the op under `sender:opId` , and broadcasts it to every peer; the sender's own copy of that broadcast is its acknowledgement. Two starting points: the newest snapshot in MinIO (named by `documents.snapshot_seq`), or what F8's idle save wrote before E6.2 if that is newer; after either, `since(seq)` replays only the journal rows past that seq, and the room starts at the last replayed seq.

Then test it

1. The room builds `sequenced` with `seq + 1` before the append. When is the number actually taken, and why does it matter?

Inside `accept`, after the append resolved: a failed write leaves no gap and the next op gets the same number

When the broadcast is sent, so peers see numbers in order

When the client's `baseSeq` catches up

When `sequenced` is built: the journal needs it to write the row

Under the reversed order a failed write would move the room one ahead of the table, and the next append would break `op_journal_seq` .

2. Both unique constraints fail with `SQLSTATE 23505`. How does `append` tell a resend from a rival writer, and what does it do for each?

By comparing the op's JSON with the existing row

By the constraint's *name* on the error: `op_journal_op` is a resend and the original row is returned; `op_journal_seq` is a rival and the error propagates

By retrying the insert and seeing which one fails twice

It cannot: both are refused as unavailable

Measured in `learning-tests/postgres` : the driver exposes `err.constraint` . `"constraint" in err` narrows the type so the compare compiles without a cast.

3. Why is the journal's dedupe key `(document, actor, opId)` and not `(document, opId)` ?

Postgres requires three columns in a unique key on a partitioned table

So that the AI's runs get their own numbering

Every broadcast shows every `opId` to every peer, so another peer could send one as its own; a bare-`opId` key would call that a resend and answer with the other peer's op

Because uuids collide often enough to matter

The database test proves two senders may share an `opId` and both rows are written.

4. A journal append fails. In what order do peers hear about it, and what does a read-only room acknowledge?

One status to every peer first, then the refusal to the sender; while read-only it acknowledges nothing: not a fresh op, not a no-op, not a resend it remembers

The refusal to the sender first, then a status to the others

It acknowledges remembered resends from memory, since those were durable

Only the sender hears; the others find out from their own next op

Even an answer from memory is an acknowledgement. The order lets the sender's client treat the refusal as "hold", not "resync".

5. What does `recover()` do before it tells peers "writable", and why is a bare health probe wrong?

It clears the room's memory so no stale answer is given

It reloads the document from the newest snapshot

It re-sends every refused op itself

It replays `since(seq)` through `accept`: an append that landed although it threw, or a rival's row, is applied and announced; a probe would go writable with the room behind the journal and the next append would break `op_journal_seq`

"The database answers" and "the room agrees with the database" are different facts. The test with `committedAnyway` is the lost-reply case.

6. Why is the snapshot written to MinIO *before* `documents.snapshot_seq` is updated, and what does `where snapshot_seq < $3` add?

Postgres would reject the update while the object is still uploading

So the pointer never names a missing object: a crash between the two leaves an orphaned object, not an unopenable document; the < keeps the pointer monotonic when a slow older write finishes late

Because MinIO is faster than Postgres

So that the listing can find the newest object

A 412 on the put is treated as success: the same journal prefix gives the same document.

7. A snapshot fails to write. What happens?

The room goes read-only until MinIO answers

The document is marked corrupt (4500)

It is logged and not retried; the journal holds every accepted op, so the next open replays more rows and loses nothing

It is retried every second until it lands, as F8's save was

F8 had to retry for ever because the room's memory was the only copy. That is no longer true, so its retry loop was deleted with it.

8. The kill-sync chaos run reports PASS. What must its ledger have contained for that PASS to mean anything?

One op from each kind of peer

At least 100 ops, so the statistics are meaningful

At least one op acknowledged before the fault and at least one in flight at it; otherwise the vacuity guards report the run as proving nothing

A snapshot taken during the run

SPEC §4a: every `always` needs a `sometimes`. The kill is synchronous so the burst is guaranteed in flight.

Next: Lesson 7, after epic 7. Multi-node: a lease per document so exactly one node owns a room, a fencing token checked by the journal append so a node paused past its lease cannot write, and clients that follow the room to its new node through `/session`; the ledger from this chapter, unchanged, checks the failover.